

JARVIS for HVAC: LLM-based Question-Answer Framework for Sensor-driven HVAC System Interaction

SUNGMIN LEE, Yonsei University, South Korea

JOONHEE LEE, Yonsei University, South Korea

MINJU KANG, Yonsei University, South Korea

SEUNGYONG LEE, LG Electronics Inc., South Korea

DONGJU KIM, LG Electronics Inc., South Korea

JINGI HONG, LG Electronics Inc., South Korea

JUN SHIN, LG Electronics Inc., South Korea

PEI ZHANG, University of Michigan, Ann Arbor, United States

JEONGGIL KO*, Yonsei University, South Korea

Question-answering (QA) interfaces powered by large language models (LLMs) present a promising direction for improving interactivity with HVAC system insights, particularly for non-expert users. However, enabling accurate, real-time, and context-aware interactions with HVAC systems introduces unique challenges, including the integration of frequently updated long-term sensor data, domain-specific knowledge grounding, and coherent multi-stage reasoning. In this paper, we present *JARVIS*, a two-stage LLM-based QA framework tailored for sensor data-driven HVAC system interaction. *JARVIS* employs an **Expert-LLM** to translate high-level user queries into structured execution instructions, and an **Agent** that performs SQL-based data retrieval, statistical processing, and final response generation. To address HVAC-specific challenges, *JARVIS* integrates (1) an adaptive context injection strategy for efficient HVAC and deployment-specific information integration, (2) a parameterized SQL builder and executor to improve data access reliability, and (3) a bottom-up planning scheme to ensure consistency across multi-stage response generation. We evaluate *JARVIS* using real-world data collected from two commercial HVAC systems and a ground truth QA dataset curated by HVAC experts to demonstrate its effectiveness in delivering accurate and interpretable responses across diverse queries. Results show that *JARVIS* outperforms baseline implemented with modern off-the-shelf LLMs and ablation variants in both automated and user-centered assessments, achieving high response quality and accuracy.

CCS Concepts: • **Information systems** → **Information retrieval**; • **Human computer interaction (HCI)** → **Natural language interfaces**.

Additional Key Words and Phrases: Interactive data retrieval; Heating, ventilation, & air conditioning; Large language model

*Corresponding author.

Sungmin Lee, School of Integrated Technology and BK21 Graduate Program in Intelligent Semiconductor Technology, College of Computing, Yonsei University, South Korea, i.am.sungmin.lee@yonsei.ac.kr; Joonhee Lee, School of Integrated Technology and BK21 Graduate Program in Intelligent Semiconductor Technology, College of Computing, Yonsei University, South Korea; Minju Kang, School of Integrated Technology and BK21 Graduate Program in Intelligent Semiconductor Technology, College of Computing, Yonsei University, South Korea; Seungyong Lee, LG Electronics Inc., South Korea; Dongju Kim, LG Electronics Inc., South Korea; Jingi Hong, LG Electronics Inc., South Korea; Jun Shin, LG Electronics Inc., South Korea; Pei Zhang, Department of EECS, University of Michigan, Ann Arbor, United States, peizhang@umich.edu; JeongGil Ko (corresponding author), School of Integrated Technology, College of Computing, Yonsei University, South Korea, jeonggil.ko@yonsei.ac.kr. For correspondence, please contact JeongGil Ko and Pei Zhang.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 2474-9567/2026/6-ART51

<https://doi.org/10.1145/3810210>

ACM Reference Format:

Sungmin Lee, Joonhee Lee, Minju Kang, Seungyong Lee, Dongju Kim, Jingsi Hong, Jun Shin, Pei Zhang, and JeongGil Ko. 2026. JARVIS for HVAC: LLM-based Question-Answer Framework for Sensor-driven HVAC System Interaction. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 10, 2, Article 51 (June 2026), 36 pages. <https://doi.org/10.1145/3810210>

1 INTRODUCTION

The sensor-driven segment of the smart building HVAC (Heating, Ventilation, and Air Conditioning) industry, encompassing applications such as fault diagnosis and energy-efficient control, is projected to represent a significant area of growth, with an estimated market value of \$8.31 billion by 2029 [54]. These systems typically require advanced domain expertise and manual effort to query internal databases and interpret time-series sensor data. To support interaction with non-expert users, enabling such capabilities through a question-answering (QA) interface offers substantial benefits—providing an interactive, user-friendly, and personalized experience [70].

Common approaches for building QA systems include deploying trained human experts or implementing rule-based bots [46, 50]. While human experts offer accurate and context-aware responses, they suffer from limited scalability and cost-effectiveness, making them unsuitable for continuous or large-scale support. Rule-based systems, in contrast, rely on manually crafted templates or decision trees to produce fixed responses. While they provide low-latency and predictable behavior, they struggle with the ambiguity and variability of natural language, especially when user queries span multiple entities, time windows, or vague contextual expressions [17, 20, 23, 35, 57, 60]. These limitations motivate the use of large language models (LLMs), which combine the scalability of automation with the adaptability of natural language understanding [21]. LLMs can generalize across diverse queries and support novel user intents without requiring continual rule engineering or expert oversight, making them particularly well-suited for dynamic, sensor-rich environments like HVAC.

When generating responses, an LLM-based HVAC QA framework must ground its outputs in real-time sensor data produced by HVAC systems. This requirement renders conventional approaches based on statically integrated databases fundamentally unsuitable, as they cannot accommodate the dynamic and frequently updated nature of operational sensor streams. Since LLMs are trained on static corpora and cannot be retrained in real time, the framework must retrieve and inject up-to-date data through alternative methods such as prompt-based augmentation [23, 28, 30, 47, 71].

In practice, HVAC sensor data are stored in structured databases that accumulate dense, long-horizon time-series records over time. Typical HVAC queries often involve extended temporal scopes, ranging from days to several months or even a full year. However, directly feeding such large data streams into an LLM is computationally infeasible due to memory and context window limitations [67]. Instead of loading the entire dataset, the system must selectively retrieve only the data segments relevant to the user’s query. This motivates the use of structured query interfaces, typically realized through SQL [5, 31, 42], for efficient, database-oriented retrieval.

Once retrieved, sensor data often require domain-specific preprocessing, such as statistical summarization, temporal filtering, or condition-based selection, to derive semantically meaningful representations suitable for language model consumption. Such preprocessing is essential for maintaining contextual accuracy, reducing prompt length, and mitigating the limitations of LLMs in performing arithmetic or logical operations [2, 6, 29]. Together, these observations highlight the need for a tightly integrated pipeline that couples structured data retrieval, HVAC-specific analytics, and natural language-based interaction to produce coherent, truthful, and actionable responses.

To support real-time QA interactions for HVAC systems while addressing the aforementioned requirements, we introduce *JARVIS*, a QA framework tailored for data-driven HVAC applications. *JARVIS* employs two distinct LLMs in a modular architecture inspired by the master-worker paradigm. Specifically, *JARVIS* consists of two sequential components: (i) an **Expert-LLM**, which interprets user requests and generates a set of precise execution instructions grounded in HVAC-specific domain knowledge; and (ii) an **Agent**, which carries out a multi-stage

answering process that includes SQL-based data retrieval, Python-based data processing, and LLM-based response generation, based on the Expert-LLM's instructions.

While being a promising approach, designing *JARVIS* introduces several key challenges. First, translating vague, high-level user requests into precise, low-level execution instructions suitable for the HVAC domain requires the Expert-LLM to incorporate both deep HVAC domain knowledge and deployment-specific context. Effectively integrating such specialized knowledge into general-purpose LLMs, while balancing data requirements and computational cost, remains an open challenge [19, 35, 62, 65, 69]. To address this, we classify contextual information based on its versatility and representation length, and adopt an *adaptive context injection* strategy that selects and integrates relevant context per category.

Second, fine-tuning LLMs for SQL query generation is known to be challenging, resource-intensive, and often brittle [17, 41, 73]. To overcome this, we introduce a *parameterized SQL builder* that constructs SQL statements from a set of required parameters. This component abstracts away redundant boilerplate syntax, enabling the Expert-LLM to focus on accurately identifying query conditions rather than managing complex SQL syntax.

Note that the multi-stage answering process requires the Expert-LLM to generate structured (and sequential) instructions that carefully guides each subsequent step toward producing a question-relevant final response, while avoiding unnecessary or misaligned intermediate goals. To maintain coherence across consecutive instructions, we adopt a *bottom-up planning* approach, in which the Expert-LLM first defines an expected answer template and then plans preceding the querying and processing steps accordingly.

We evaluate *JARVIS* using sensor data collected from two real-world commercial-scale HVAC deployments and a ground truth QA dataset curated by HVAC experts. Our results show that *JARVIS* consistently outperforms ablation variants, widely-used off-the-shelf cloud-based LLM baselines, and other local LLM baseline configurations across multiple response quality metrics, including cohesiveness, helpfulness, truthfulness, and faithfulness. Through both automated evaluations using an *LLM-as-a-Judge* [27, 34, 56] and a real-world user study, we demonstrate that each component in *JARVIS* contributes meaningfully to the overall quality of system responses.

Specifically, we make the following contributions in this work:

- We categorize HVAC-specific and deployment-specific contextual information required for QA applications, and propose an adaptive context injection strategy for data- and cost-efficient integration into general-purpose LLMs.
- We achieve LLM-driven sensor data retrieval through a parameterized SQL builder and executor that leverages template-based query construction, to reduce fragility and improve reliability.
- We introduce *JARVIS*, a two-stage LLM-based QA framework for data-driven HVAC applications, leveraging bottom-up planning and variable alignment to ensure coherent multi-step execution.
- We evaluate *JARVIS* using sensor data collected from two real-world, building-scale HVAC deployments and an expert-curated QA dataset to show its effectiveness as an automated HVAC QA system.

2 BACKGROUND AND RELATED WORK

2.1 LLM in HVAC Systems

Recent studies have begun to explore the use of large language models (LLMs) in HVAC applications. Lu et al. [35] evaluated commercial LLMs such as GPT-4 and LLaMA for their HVAC-related knowledge and reasoning capabilities using the ASHRAE Certified HVAC Designer examination [3], a professional certification used to assess HVAC expertise across a wide range of applications. Their results revealed that general-purpose LLMs lack sufficient domain knowledge, suggesting the need to inject HVAC-specific knowledge and application context into LLMs during inference.

Prior works have primarily adopted two approaches for incorporating such domain knowledge. Zhang et al. [69] developed an HVAC fault-diagnosis assistant by fine-tuning a general-purpose LLM using labeled sensor

data from building systems. As emphasized in their work and in other LLM literature [4, 16, 23, 52, 63, 71], fine-tuning integrates rich and complex knowledge directly into model weights. This approach supports the reliable injection of large context volumes, but lacks adaptability to post-deployment updates and requires a costly, domain-specific training dataset.

Alternatively, some studies have employed prompt engineering, which involves prepending relevant contextual information to the original query at inference time. Jiang et al. [20] utilized prompt engineering to construct an LLM-based assistant for automated building energy modeling. This approach enables rapid prototyping and the incorporation of up-to-date information, but typically suffers from higher inference latency and reduced accuracy when processing complex knowledge or long-context inputs [2, 6, 22, 47]. Moreover, as we will later show in our preliminary studies, HVAC sensor data are often large and temporally dense, making direct prompting impractical.

Beyond knowledge integration for analysis and diagnosis, recent work has also explored the use of LLMs in control-oriented HVAC pipelines, such as supporting or augmenting model predictive control (MPC) frameworks [1, 36]. These studies highlight the expanding role of LLMs in HVAC systems, spanning both analytical tasks and operational control. Taken together, prior work suggests that successfully deploying LLMs in HVAC environments requires careful integration of domain knowledge, sensor data, and system objectives, while balancing expressiveness, adaptability, and computational efficiency.

2.2 Incorporating Up-to-Date Time-Series Sensor Data from Tabular Databases

User-interactive HVAC applications rely on leveraging real-time time-series sensor data collected from building sensors often stored in databases. Since real-time data is not accessible during the fine-tuning stage, LLMs for these applications require an external framework that retrieves relevant sensor data at inference time and augments them via prompting (e.g., retrieval-augmented generation (RAG)) [12, 13, 23, 51, 71].

As noted in Section 2.1, LLM performance tends to degrade and inference cost increases with longer inputs. Therefore, it is crucial to only exploit data that is both highly relevant and concise in the augmentation process, avoiding unnecessary or irrelevant information [22]. While some previous work such as TaskSense attempt to access external data via external API calls [33], a more general approach for augmenting LLMs with time-series sensor data stored in relational databases would involve translating user input into SQL queries (text-to-SQL [17, 41, 66, 73]) to identify and leverage the most relevant sensor information. Biswal et al. [5] introduced a two-stage LLM architecture, employing a front-end LLM to perform text-to-SQL translation and a back-end LLM to generate natural language responses based on the retrieved data.

Our work acknowledges the observations made by Biswal et al., and targets to address several key limitations and challenges. First, the LLMs that Biswal et al. use for both stages are general-purpose LLMs. Thus, their system lacks HVAC-specific domain knowledge and therefore is likely to underperform in HVAC-specific scenarios. We address this by injecting HVAC-specific and deployment-context knowledge to the front-end model, enabling it to relay essential domain information to the general-purpose back-end model for more accurate answer generation.

Second, our preliminary experiments reveal that directly generating complete SQL statements (i.e., boilerplate syntax, nested queries, and multiple joins) negatively impacts the system robustness due to inconsistencies and errors in the generated queries themselves. To mitigate this, we propose the parameterized SQL builder where the LLM produces an abstract query plan or key-value directives, and a rule-based module then completes the final SQL statement, ensuring syntactic and semantic correctness.

Third, general-purpose LLMs typically lack the mathematical and logical reasoning capabilities required to analyze raw HVAC sensor data and become computationally expensive when a long, high-frequency time-series streams are input directly. To address this limitation, we introduce a Python-based data processing pipeline situated between the data retrieval and answer generation stages, similar to the structured intermediate processing

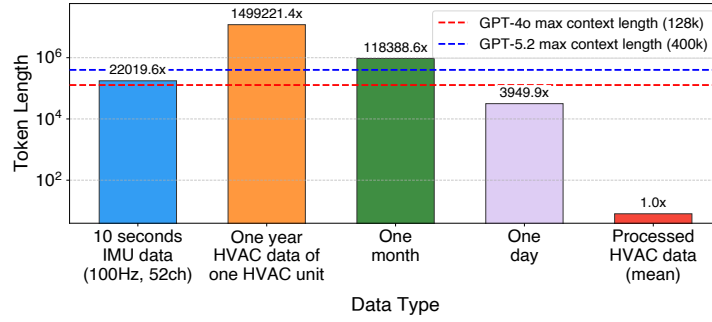


Fig. 1. Required token length when prompting sensor data into LLM context (lower the better).

proposed by Xiao et al. [61]. This module performs essential computations such as statistical summarization, temporal aggregation, and result formatting, thereby enhancing the downstream reasoning capabilities of the LLM while significantly reducing prompt length and complexity. By offloading domain-specific analytics to an efficient procedural backend, this approach improves numerical accuracy and supports real-time, cost-effective system operation.

3 PRELIMINARY STUDY

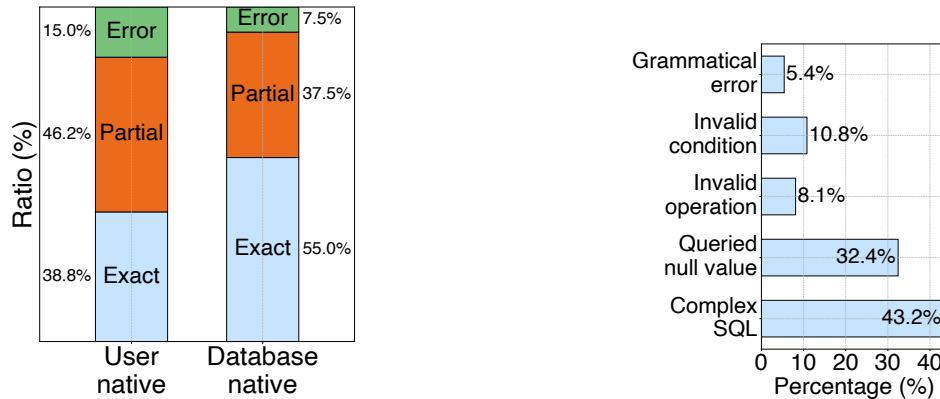
Before designing JARVIS, we conducted a preliminary investigation to identify the practical challenges of integrating real-time HVAC sensor data into LLM-based QA systems. This study aimed to quantify the computational cost and contextual constraints of directly prompting LLMs with large-scale sensor streams, and to examine the limitations of existing text-to-SQL mechanisms in time-series data environments. Through this analysis, we establish two critical design motivations for JARVIS: (i) the necessity of selective and context-aware data querying to remain within model context limits, and (ii) the importance of robust intermediate reasoning and structured data processing to ensure reliable query execution and response accuracy.

3.1 Cost of Direct Sensor Data Prompting

We begin by discussing unique challenges involved in incorporating external HVAC sensor data streams into an LLM-based QA framework. This analysis motivates the necessity of selective data querying and preprocessing before answer generation.

To illustrate the limitations of direct sensor data prompting, we measured the token length incurred when serializing representative sensor data streams as plain text and encoding them using a LLaMA-style tokenizer [14]. Specifically, we considered five scenarios, each corresponding to a plausible sensor-LLM integration strategy: (1) a 10-second segment of IMU data sampled at 100 Hz across 52 channels from the PAMAP2 benchmark [44], (2-4) a one-year, one-month, and one-day HVAC dataset collected from a single unit in a commercial building, and (5) a preprocessed HVAC dataset containing aggregated statistics, such as daily average temperature. In each case, the numeric values and timestamps were concatenated as text and tokenized using the LLaMA tokenizer.

As Figure 1 shows, directly exploiting one-year, one-month, and one-day HVAC data streams result in token lengths approximately 3.9×10^3 to 1.5×10^6 times greater than an approach exploiting processed data. While the use of one-day query remains within GPT-4o's nominal 128K token limit, both the one-month and one-year queries vastly exceed this limit, and even the one-day case leaves only limited room for reasoning and response generation. Similarly, even a short 10-second IMU window produces approximately 6.6×10^4 times more tokens due to its high sampling frequency and floating-point precision across multiple sampling channels. This observation highlights a practical limitation that when users pose a common and intuitive query such as “What



(a) Query success rate with respect to input taxonomy (b) Reasons of failure (partial + error) for database-native inputs
 Fig. 2. Failure analysis of generic text-to-sql operations in an HVAC scenario.

was the average temperature in my room this month?”, it is infeasible for either users or QA systems to simply prompt the LLM with all raw sensor readings from the corresponding time span. Plots in Figure 1 imply that even such seemingly reasonable requests lead to token volumes that substantially exceed practical context limits without explicit data selection or aggregation, resulting in context overflow and prohibitive computational costs. As a result this suggests that directly embedding raw sensor streams into prompts is an unsuitable approach for real-time interaction, particularly for long-term deployments or high-frequency sensing environments, a bottleneck also noted in prior work [67].

To address this scalability challenge, we put focus on two design principles that guide our system architecture. First, sensor data should be selectively retrieved based on query relevance, motivating an SQL-based retrieval layer; and second, raw sensor measurements should be preprocessed into compact numerical descriptors, such as statistical summaries, trends, or anomaly scores, prior to prompting. This structured pipeline not only ensures accurate and context-bounded reasoning, but also enhances numerical reliability and supports real-time responsiveness—key requirements for practical LLM-based HVAC applications.

3.2 Failure Analysis of Text-to-SQL in Time-Series HVAC Data

Next, we analyze failure cases of text-to-SQL operations to better understand the challenges discussed earlier and motivate the design choices in our proposed approach. The analysis was conducted using a simple toy configuration as follows. We used one year of HVAC time-series sensor data collected from three rooms from a real-world large-scale building deployment, covering two modalities: room temperature and user-desired temperature. We note that this data is collected from a commercial deployment and we have gained the proper permissions to analyze and publish analytical results using the data. Based on this time-series data, HVAC experts curated 80 question-SQL pairs spanning a range of user requests, from simple queries (e.g., retrieving the current temperature of a room) to more complex statistical ones (e.g., identifying the date last month when the daily average temperature was lowest).

To assess the role of language abstraction, each question (i.e., LLM query) was prepared in two taxonomic forms. First, user-native queries used natural phrasing familiar to non-expert users (e.g., “How hot is our room?”), while database-native queries directly aligned with the schema without ambiguous or missing conditions (e.g., “What is the current roomtemp of room101?”, where roomtemp and room101 correspond to column names and identifiers in the database). A LLaMA3-8B model, fine-tuned for general-purpose text-to-SQL, was used as the baseline model for this purpose.

Using this setup, we perform two different experiments. In the first experiment, we evaluated the model's ability to handle user-native taxonomies. For each user request, both the user-native and corresponding database-native version were fed into the model, and the generated SQL was compared with the expert-crafted ground truth by checking if the query results matched exactly (exact match). Ideally, the LLM should output SQL statements semantically similar to the ground truth. In case a mismatch occurs, we classify it as a partial match (including cases with no overlap). If execution failed due to syntax errors or invalid structure, we labeled it as an error. When evaluating user-native inputs, we also provided the model with the mapping between user-native and database-native taxonomies as part of the system prompt.

As shown in Figure 2 (a), the model performed significantly better on database-native queries, where queries were "intuitive" to understand. The exact match rate for user-native inputs remained below 40%, and the gap in performance suggests that even with explicit taxonomy mapping, the general-purpose text-to-SQL model struggles to bridge the abstraction gap. This suggests the need for improved approaches to interpreting natural, user-native inquiries.

In the second experiment, we investigated the causes of failure by asking four HVAC experts to debug and identify the erroneous outputs (Figure 2 (b)). From this, we identified that the most common source of failure was the generation of fragile and overly complex SQL statements, particularly those involving subqueries and multi-table joins. These queries are prone to syntactic and logical errors, especially since many text-to-SQL models are pre-optimized to generate a single comprehensive SQL per request. For statistical operations such as `argmax`, which are not natively supported in SQL, the model attempts to simulate the operation using complex constructs, further increasing the likelihood of failure.

Another significant failure cause we noticed was the improper handling of missing sensor values. As missing data is common in many real-world sensor deployments [7, 25], our dataset contained a fairly large number of null entries representing sensor failures. In these preliminary experiments we instructed the model via prompt to exclude these values during statistical computation using appropriate SQL filtering (e.g., `IS NOT NULL`). Nevertheless, the model frequently failed to apply such conditions reliably.

Based on these observations from our preliminary study, we focus on two major design changes. For handling missing values, we use post-generation boilerplate logic to automatically append filtering conditions that exclude null elements, improving robustness without burdening the model. To address the issue of complex SQLs, rather than relying on the model to generate a monolithic query, we instead train a custom LLM to generate a sequence of short, modular SQLs that can be combined to generate a final result. This approach not only reduces the chance of syntax errors but also increases reusability and consistency, as common query patterns emerge across simpler blocks. Together, these solutions motivate a parameterized SQL generation framework, where the model is responsible for specifying high-level query intentions and leaves repetitive or syntax-heavy component (e.g., joins, filters) generation to deterministic logic.

Finally, we also address the common failure regarding the incorrect use of SQL's built-in statistical functions. To support operations not available in SQL or prone to misinterpretation, we use Python-based post-processing within the system pipeline to handle such computations. This integrated pipeline architecture ensures accurate statistical analysis and expands the range of supported user queries and operations.

4 JARVIS DESIGN

Based on the insights from our preliminary study, we propose *JARVIS*, an LLM-based HVAC QA framework. In this section, we first outline the design goals of *JARVIS*, followed by an overview of its architecture and details on key modules.

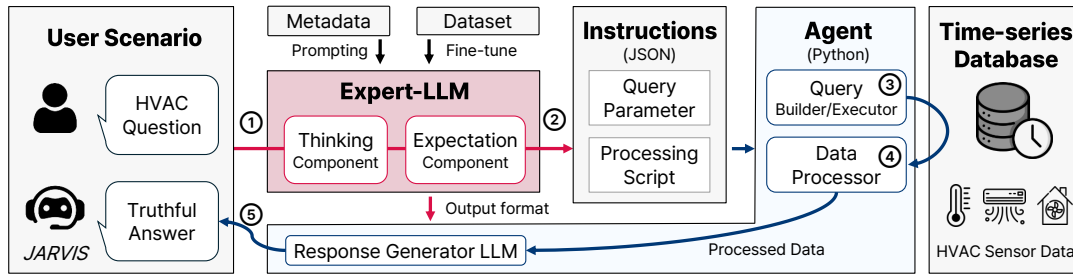


Fig. 3. Operational flow of JARVIS.

4.1 Design Goals

We define four core design goals that guide the development of JARVIS to address the challenges in Section 3.

G1. User-Native Query Interpretation in HVAC Context. HVAC users typically describe situations using everyday language, which differs significantly from the structured syntax of the underlying system. Phrases such as “Our room feels hot” encode both implicit intent and contextual cues, but not explicit variables. The framework must accurately interpret this user-centric grammar and map it to system-level semantics, ensuring that responses remain aligned with how users perceive and articulate their environments.

G2. Precise and Contextual Retrieval of HVAC Sensor Data. HVAC systems generate high-frequency, long-term time-series data that are continually appended to structured databases. Feeding the entirety of this data into an LLM would quickly exceed context limits and degrade response accuracy. The system must therefore identify and summarize only the most relevant portions of the data stream, ensuring that answers remain both contextually grounded and computationally efficient.

G3. Coherent Multi-Stage Reasoning. As discussed in Section 3, single-shot text-to-SQL generation is fragile, especially when user queries involve vague language, extended temporal scopes, or require statistical computation. To ensure stable and interpretable responses, the system must decompose the reasoning process into well-defined stages that connect query interpretation, data processing, and answer formulation. Preserving coherence and goal alignment across these stages ensures that each reasoning and execution step remains consistent with the intended answer structure and overall system objective.

G4. Efficient, Adaptable, Scalable, and Robust Deployment. Real-world HVAC environments vary widely in building scale, sensor placement, data schema, and computational capacity. The system must generate accurate responses efficiently, adapt to heterogeneous configurations without requiring retraining, and scale from individual HVAC units to large multi-zone buildings while maintaining consistent reasoning quality.

4.2 Design Overview

As an LLM-based HVAC QA framework, JARVIS adopts a master-worker architecture and executes a five-stage answering pipeline, as illustrated in Figure 3. At the core of this design, an **Expert-LLM** (master) is responsible for interpreting user queries and orchestrating the overall reasoning process (①). Specifically, the Expert-LLM generates a structured set of *execution instructions* (②) that decompose the user request into discrete subtasks required to formulate a response.

Following these instructions, JARVIS’s **Agent** (worker) executes three core operations: (i) querying real-time sensor data from the HVAC database using SQL (③), (ii) performing application-specific data processing, such as statistical aggregation or anomaly detection (④), and (iii) generating a final natural language response using a general-purpose LLM (⑤).

Table 1. Ideal/target context injection method with respect to contextual characteristics in a domain-specific QA system (‘-’: Not necessary, ‘O’: Applicable, ‘X’: Not applicable).

	Semantic context		Sensor context	
	HVAC-common knowledge	Deployment and user specific context	Small-scale sensor data	Large-scale sensor data
Occurrence time	Before deployment	Before request	In-situ (real-time)	In-situ (real-time)
Context length	Long	Short	Short	Long
Pre-training	X	X	X	X
Fine-tuning	O	X	X	X
Direct Prompting	-	O	O	X
Processed Prompting	-	-	O	O
Examples	Presuppositions	Per-user preference	IMU sensor data	One-year HVAC data

The final response is produced by a separate, lightweight LLM module, guided by the processed data and a set of application-specific *answering guidelines* that promote consistency, numerical faithfulness, and user-oriented tone. This modular and staged architecture ensures that JARVIS can adapt to continuously generated dynamic sensor data, operate within LLM context limits, and maintain consistency across each reasoning step.

4.3 Adaptive Context Injection

As discussed in Section 2, incorporating contextual information into LLMs is essential for effective HVAC QA, but prior work typically relies on a single injection method (i.e., prompting or fine-tuning) without proper justification. In contrast, we classify the context into four types based on versatility and representation length, and apply an *adaptive injection* strategy tailored to each category (Table 1).

HVAC-common knowledge, such as domain-specific reasoning patterns (e.g., time-series data analysis) or linguistic conventions in user queries, is deployment-agnostic and stable across scenarios. This type of context often requires lengthy descriptions and diverse examples, making it best suited for fine-tuning. Once embedded into model weights, this knowledge remains robust after deployment.

Deployment-specific and user-specific contexts, including sensor layout maps or individual user preferences, are dynamic and vary by installation. These inputs are typically compact (e.g., configuration files or metadata) and can be efficiently injected via prompting at inference time.

Small-scale sensor data from short time-series windows, are concise and adaptable, making them ideal for direct prompting. However, **large-scale sensor data**, such as high-frequency or multi-month streams, can exceed the input context length. Additionally, LLMs often struggle to perform necessary statistical analysis or anomaly detection directly on raw data. For this reason, we propose to algorithmically preprocess or summarize such data prior to injection.

Overall, JARVIS adopts this adaptive context injection strategy by categorizing input contexts according to the above framework and selecting the optimal injection method to balance answer quality and inference cost. In particular, we fine-tune the front-end Expert-LLM with HVAC-common knowledge and dynamically provide it with deployment- and user-specific contexts via prompting. For sensor data, we follow the execution instructions generated by the Expert-LLM to retrieve and process relevant subsets. Large-scale sensor data are algorithmically processed and summarized before being passed as input, thus mitigating context length limitations while preserving essential information.

Throughout the paper, we will refer to the HVAC-common knowledge and deployment- and user-specific contexts as *semantic context* and the both sensor data types as *sensor context*.

4.4 Expert-LLM

As demonstrated in our preliminary study and through prior work, using a text-to-SQL model alone to understand user requests and generate executable queries is insufficient for HVAC applications. Instead of proposing incremental improvements to existing text-to-SQL approaches, *JARVIS* introduces a specialized HVAC-expert LLM, built on top of a general-purpose LLM, and trained to serve a broader and more semantically rich role.

Unlike traditional text-to-SQL models, which focus solely on converting natural language into executable SQL statements, with the HVAC-expert LLM acting as a translator and a strategist between the user and the system agent. Specifically, its responsibilities are twofold: (1) to incorporate semantic context and accurately interpret user-native requests; and (2) to plan and direct the system’s response pipeline by generating structured, executable instructions.

The operation of the Expert-LLM begins with a natural language question provided by the user. Upon receiving the question, the model first interprets its meaning in the context of relevant semantic and contextual information (e.g., HVAC-specific terminology, user preference). It then produces a set of structured execution instructions formatted in JSON, which guide the downstream components, including data retrieval, processing, and response generation, toward producing an truthful and context-aware answer.

4.4.1 Semantic Context Injection. To enable the HVAC-expert LLM to interpret user-native queries and plan appropriate actions, semantic context must be integrated (Section 2.1). In *JARVIS* we employ both fine-tuning and prompting for this context injection process based on the adaptive context injection strategy.

For HVAC-common knowledge, we fine-tune the base LLM using an expert-curated dataset. While the scope of this knowledge may vary across deployments, we identify a set of generalizable knowledge components that are broadly applicable, such as presuppositions underlying HVAC-related queries, preferred answering strategies, typical data query patterns, and standard data processing procedures. This process also teaches the Expert-LLM to produce well-structured, syntactically valid JSON instructions.

For deployment- and user-specific contexts, we use prompting at inference time. Specifically, these contexts are structured as a key-value dictionary referred to as *metadata*, comprising two core components: (1) a mapping between user-native and database-native taxonomies for spatial information and sensing modalities, and (2) inference-time environmental factors such as the current date and time. We note that this structure is extensible and can be customized based on application-specific needs.

4.4.2 Thinking Component: Intermediate Thought Generation. Prompting typically imparts static rules and constraints to an LLM, whereas fine-tuning enables the model to emulate expert behavior, capturing both explicit patterns and implicit reasoning strategies. Prior work has shown that generating intermediate reasoning steps via the chain-of-thought (CoT) approach, can significantly enhance the reasoning capabilities of autoregressive LLMs [10, 15, 49, 58]. This not only improves inference accuracy but also increases transparency by exposing the model’s decision-making process.

To leverage these benefits, *JARVIS* incorporates a dedicated thinking component that guides the Expert-LLM to explicitly emulate human-like reasoning through intermediate “thoughts” prior to instruction generation. A human expert answering an HVAC-related query typically interprets the question within its context, identifies the relevant scope of data and tools, develops a high-level plan, and finally converts this plan into actionable instructions. In *JARVIS*, the thinking component addresses the first three stages, while the final translation into structured instructions is handled by the instruction generation module (see Sec. 4.4.4).

Within the Thinking component, the Expert-LLM focuses on three core functions. First, it resolves ambiguous or user-native terminology by aligning it with the database-native taxonomy defined in the metadata. Second, it infers and fills in missing or underspecified conditions based on presuppositions learned during fine-tuning, resulting in a more explicit and actionable reformulation of the user query. Third, it outlines a plan for answering

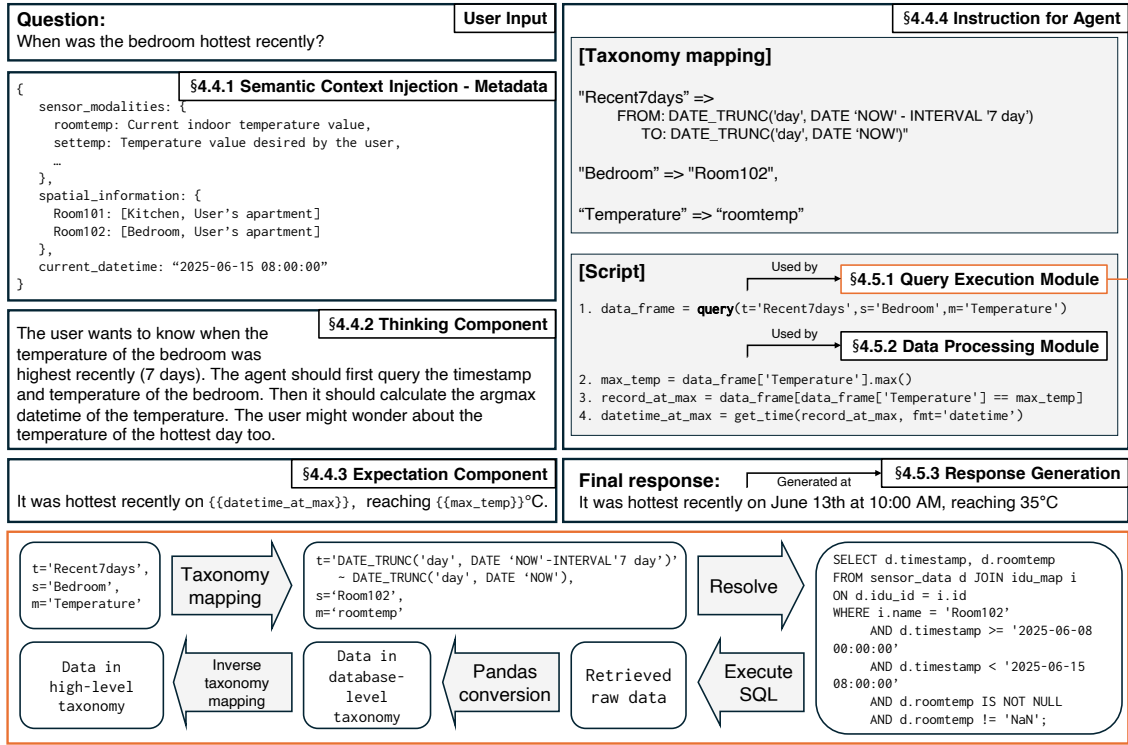


Fig. 4. Sample operation of Expert-LLM generating execution instructions based on user input.

the query, including necessary data retrieval and processing steps, to ensure all relevant information is collected before response generation begins.

To support these capabilities, we manually curate training examples using the CoT methodology, capturing each of these reasoning processes. We present an example of such training in Figure 4.

4.4.3 Expectation Prediction. Upon generation of the Thinking component, the Expert-LLM produces the User Expectation Component, serving three primary purposes.

First, the User Expectation component anticipates additional information that users may implicitly expect. For example, given the request “When was our room coldest today?”, the Expert-LLM infers that the user likely also wants to know the actual temperature at that time. By predicting such latent expectations based on HVAC-common knowledge, JARVIS enriches the user experience by proactively including useful supplemental details.

Second, it guides the Response Generation LLM with a context-aware answer format. Even when the same data is available, the preferred tone, level of detail, and response structure can vary depending on HVAC norms or individual preferences. The user expectation component provides an explicit example format that the response generator can follow, serving as a direct and lightweight injection mechanism to convey contextual instructions.

Third, the User Expectation component defines the data that must be queried and processed by the Agent before initiating answer generation. For instance, in response to the query “Has the temperature in Room 101 been unstable recently?”, the Expert-LLM may set the expectation to include a time window and a summary of daily fluctuations; thus, enabling the system to prepare all necessary data in advance. We provide an example of such a user expectation component training sample in Figure 4.

4.4.4 Generating Instructions for the Agent. After completing the thinking and expectation components, the Expert-LLM proceeds to generate executable instructions for the Agent. As the example in Figure 4 shows, these instructions are structured in JSON format, allowing the Agent to parse and execute them deterministically. Note that the reliability of *JARVIS* partially depends on the Expert-LLM’s ability to generate syntactically correct and semantically meaningful JSON. To ensure this, we combine fine-tuning with lightweight prompt-based guidance, which together enable the model to produce consistently well-formed outputs.

The specific structure and contents of these instructions will be detailed in the following sections, alongside an introduction to the Agent and its role in the overall execution pipeline.

4.4.5 Bottom-up Operation Planning. In general reasoning processes, the overall workflow typically proceeds from understanding a problem to deriving the final answer. In contrast, *JARVIS* adopts a bottom-up planning approach, where the Expert-LLM first configures the desired outcome through the Expectation component and then works backward to determine the contextual information and execution steps required to achieve it.

After forming a high-level reasoning trace in the Thinking phase, the model predicts the user’s intended goal and preferred answer structure in the Expectation phase. This explicit goal then serves as the anchor for subsequent planning. Based on the goal, the model identifies the necessary contextual bindings such as relevant sensors, spatial or temporal scopes, and database fields, which we refer to as Mapping in the later section. Finally, it generates a Script, which is a structured set of executable instructions that realizes the Expectation goals.

By grounding the reasoning on a clearly defined outcome and constructing the plan in this bottom-up manner, *JARVIS* ensures that each mapping and scripting step directly supports the intended result. This goal-driven ordering improves robustness across multi-step decoding and enhances consistency between reasoning, query generation, and final response.

4.5 Agent

Given the execution instructions generated by the Expert-LLM, the Agent operates on top of a Python runtime to perform the specified tasks. In *JARVIS*, the Agent is responsible for three primary operations: data querying, data processing, and response generation.

For data querying and processing, the instructions follow a simple imperative structure in the form of `variable = expression`, where the right-hand side expression is evaluated, and its result is stored in the specified variable for use in subsequent operations. For response generation, a general-purpose LLM is invoked with the answering guideline from the Expert-LLM and the queried and processed data to ensure the final user-facing response is both contextually grounded and semantically fluent.

4.5.1 Query Execution Module. As highlighted in the preliminary study (Section 3), conventional text-to-SQL approaches often fail in HVAC applications due to three key issues: (1) inability to interpret user-native taxonomies, (2) brittleness when generating complex, monolithic SQL statements, and (3) failure to include necessary boilerplate syntax, such as null-value filtering.

To overcome these limitations, *JARVIS* decouples *what to query* from *how to express it*. Rather than relying on the LLM to directly produce fully-formed SQL queries, the Expert-LLM focuses solely on generating high-level query intentions. These are then passed to a rule-based SQL generation module within the Agent, called the Query Execution Module, which constructs executable SQL statements from the LLM-supplied intent. This separation improves modularity and robustness, avoiding the fragility of deeply nested SQL generation and the inefficiencies of generating a complete query in a single inference step. The Expert-LLM can instead emit multiple small, composable instructions that collectively achieve the desired analytical objective with greater interpretability and stability.

Each instruction delivered to the parameterized query builder consists of *mapping* and *calling* components. The *mapping* component defines a key–value dictionary that aligns user-native taxonomies (refined during instruction generation) with database-native field names (c.f., taxonomy mapping in Figure 4). The *calling* component triggers an Agent-side function that performs SQL generation and execution (c.f., query function in Figure 4). This function uses the provided user-level conditions, applies the mapping to translate them into schema-compliant field names, and constructs the full SQL query, including any necessary subqueries, joins, or filtering clauses. More advanced operations can be invoked via high-level keywords, which are internally expanded by the Agent into predefined SQL templates.

The executed query returns a result in the form of a pandas DataFrame [37, 43]. While the result is initially expressed using database-native column names, the Agent automatically translates them back to user-native terminology before passing the data to the downstream processing module (Section 4.5.2). Since this “call–execute–translate” cycle is repeatable, the module supports iterative and composable querying, allowing the Expert-LLM to incrementally refine its information retrieval strategy. The bottom illustration in Figure 4 exemplifies this process. When the Expert-LLM emits the instruction query($t='Recent7days'$, $s='Bedroom'$, $m='Temperature'$), the module resolves each argument via taxonomy mapping: ‘Bedroom’ maps to the indoor unit identifier ‘Room102’, ‘Temperature’ maps to the database column `roomtemp`, and ‘Recent7days’ expands to a half-open interval representing the previous seven days. The module then constructs the SQL query by combining these resolved parameters with appropriate join clauses and boilerplate syntax.

Several embedded design constraints in the SQL generation pipeline proved essential for real-world reliability. First, temporal conditions are expressed as half-open intervals ($\geq start$ AND $< end$) to avoid overlap in daily aggregations. Second, temporal keywords such as ‘Recent7days’ and ‘yesterday’ are first expanded to static expressions containing a NOW placeholder, which is later replaced with the actual datetime via metadata injection. This eliminates errors caused by the LLM’s uncertainty about current time. Third, boilerplate clauses (e.g., table joins, null filters) are encapsulated in the module, ensuring syntactic consistency across instructions. These constraints are appended automatically, relieving the LLM from generating fragile or error-prone SQL syntax.

In summary, the parameterized SQL builder and executor design enhances the modularity, reliability, and extensibility of the data access layer. By abstracting away the low-level details of SQL syntax, schema knowledge, and error handling, this design enables the Expert-LLM to focus on high-level instruction generation. Furthermore, the functionalized structure enables system-level control (e.g., integrating a data-security middleware that can intercept and audit query calls), ensuring compliance with privacy and safety policies in HVAC deployments.

4.5.2 Data Processing Module. Once the data is queried, JARVIS’s Data Processor processes the retrieved data to extract meaningful insights based on the Expert-LLM’s instructions (c.f., Fig. 4). This design mitigates the limitations of LLMs in handling mathematical reasoning and also reduces the cost of response generation. In JARVIS, we use Python’s Pandas library [43] for data processing rather than relying on native SQL operations, which often lack the necessary functionality. Notably, more complex processing such as diagnostic modules, classifiers, or DNN-based analysis frameworks [40] can also be integrated at this stage, provided that they are implemented in Python and conform to the expected input-output interface.

4.5.3 Response Generation Using a General-Purpose LLM. With the user’s original request, the summarized and processed data, and the guidance provided by the User Expectation component, JARVIS leverages a general-purpose LLM to generate the final response. Here, the Expectation component plays a central role in shaping the tone, structure, and completeness of the output, serving as a soft template or reference for the LLM.

If any required data is missing, either due to execution failure or unavailable sensor readings, the LLM is prompted to generate a graceful fallback response instead of hallucinating untruthful responses. This may include a natural-language explanation of the partial result or an error message clarifying the reason for failure, guided

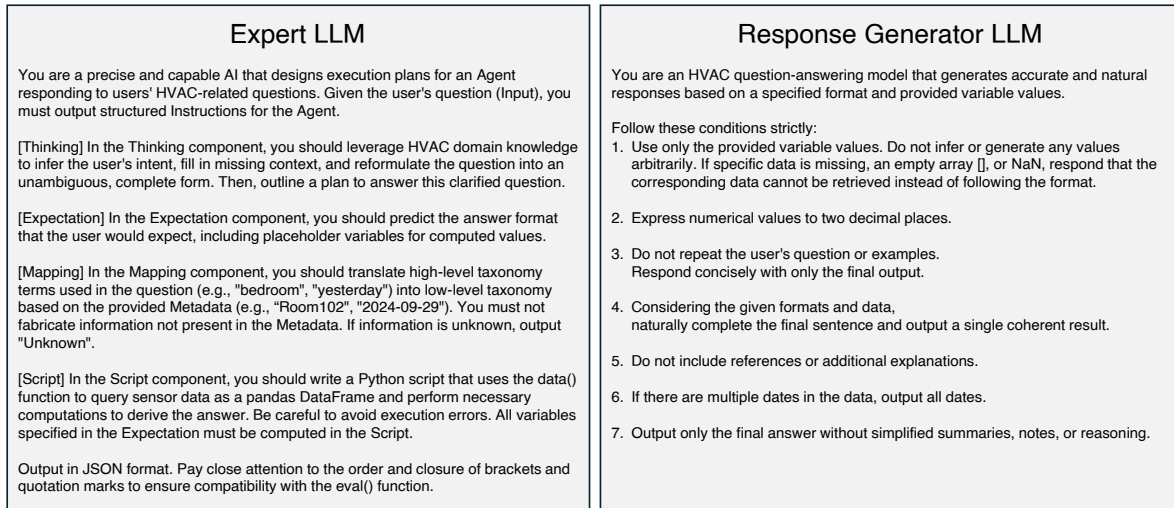


Fig. 5. System prompts used for the Expert-LLM and the response generator LLM.

by the expectation context. This flexibility ensures that the system remains robust and communicative, even when operating under uncertainty or incomplete data availability.

5 IMPLEMENTATION DETAILS

This section presents details on the implementation of *JARVIS*. Most importantly, for the Expert-LLM, we use the LLaMA3.1-8B-Instruct model [14]. Among various sized LLMs, we choose an 8B model given its practical balance between response accuracy and system latency where both are critical factors for ensuring a high-quality user experience in real-time interactive settings. Empirical results that justify this selection will be discussed in Sections 6.3 and 6.4. We fine-tune this model via low-rank adaptation (LoRA) [18] on a dataset comprising 53 (out of 107) expert-authored HVAC QA samples, selected to reflect the language patterns and operational contexts of the target deployment environment. Each sample is annotated by domain experts who directly provide intermediate reasoning (i.e., *thinking*), data retrieval and processing (i.e., *script*) as part of the ground truth. These annotations are structured as a single JSON object consisting of four fields, namely *thinking*, *expectation*, *mapping*, and *script*. During training, the model is optimized to generate the complete JSON object as a single autoregressive sequence, using a standard cross-entropy loss applied uniformly across all tokens.

As the response generator LLM, we employ LG EXAONE-3.5-7.8B LLM [45] without fine-tuning, selected for its strong performance in producing high-quality natural language responses in the target deployment language.

Figure 5 presents the system prompts used for the Expert-LLM and response generator LLM. These prompts remain identical during both fine-tuning and inference, ensuring that the models' behavior is consistent across training and deployment. The Expert-LLM prompt structures the output into four *JARVIS* design components, namely Thinking, Expectation, Mapping, and Script, while also enforcing strict JSON syntax to ensure machine-parseable outputs. Meanwhile, the response generator prompt enforces strict constraints including numerical formatting, faithful use of retrieved values, and explicit failure reporting when data is unavailable.

For our evaluations, we deploy *JARVIS* on a server equipped with an Intel Xeon Gold 6448Y 128-core CPU with 128 GB of RAM and two NVIDIA RTX 4080 Super GPUs (16 GB VRAM each), with inference optimized using NVIDIA TensorRT-LLM [38]. We note that we also report results using a NVIDIA H100 SXM GPU (80 GB VRAM) optimized using NVIDIA TensorRT-LLM for experiments that exploit the 70B model due to memory

limitations of the RTX 4080 GPUs. Our configuration reflects a practical on-premise deployment scenario, where a small number of desktop-level GPUs can be provisioned at the building or campus scale, rather than relying on specialized datacenter-grade hardware or reliable external connections, which lead to additional costs.

Finally, for time-series HVAC sensor data storage, we deployed an instance of the TimescaleDB [55], which is well-suited for managing large volumes of real-time sensor data. The data processing module in JARVIS is implemented using the Pandas library atop a Python runtime. We host this time-series database on a separate on-site server running an AMD EPYC 7443P 24-core CPU, 128 GB of RAM, and a 2 TB SSD.

6 EVALUATION

We evaluate JARVIS using data from real-world commercial-scale HVAC deployments at two large building sites. Based on this data, we constructed a QA dataset curated by HVAC experts, which serves as the foundation for our evaluation (dataset details in Sec. 6.1).

Our evaluation of LLM response quality employs five metrics and two complementary methods. The metrics consist of three judgment-based measures, cohesiveness, helpfulness, and truthfulness, as well as two RAGAS-based [11] reference-aware measures, faithfulness and answer relevancy. As evaluation methods, we adopt the *LLM-as-a-Judge* framework, in which a separate LLM assesses whether JARVIS's responses meet the expectations defined in the dataset, enabling scalable and automated evaluation. In addition, we conduct a user study with 22 participants to provide human-centered insights into system quality.

Using these metrics and methods, we compare JARVIS alternative baseline architectures, which include the use of off-the-shelf cloud LLM-based approaches (Sec. 6.3). We further examine the impact of different base models for the Expert-LLM in Section 6.4, conduct component-level ablation studies of JARVIS in Section 6.5, and evaluate system-level query accuracy and inference latency in Sections 6.7 and 6.8.

6.1 HVAC QA Dataset Collection

6.1.1 HVAC Data Collection Environment. We collect and utilize one year of HVAC time-series sensor data, sampled at one-minute intervals, from two real-world large-scale building deployments. These deployments consist of a high school building and a residential complex with 156 and 984 HVAC units, respectively¹. The school building features regular occupancy patterns and temporally synchronized schedules across classrooms, whereas the residential complex exhibits highly diverse, user-driven usage behaviors across individual households.

All HVAC sensor readings are stored in a tabular database to support structured querying and efficient retrieval. Specifically, the database comprises two tables: (i) a primary sensor data table containing 78 columns of time-series measurements, and (ii) a spatial mapping table that links indoor unit identifiers to deployment-specific metadata. As summarized in Table 2, sensor attributes are grouped into five categories: identifiers and timestamps for hierarchical addressing and temporal reference; environmental measurements (e.g., room temperature, set temperature, humidity); operational states and control signals (e.g., power status, mode, fan speed); user configuration parameters for temperature control; and diagnostic indicators such as error codes. This structure introduces nontrivial query complexity, as answering user questions often requires spatial joins across tables, temporal filtering over multiple timestamp fields, categorical normalization of operational states, and explicit handling of missing values due to sensor dropouts common in practice.

To simulate realistic user interactions and evaluate the system's flexibility, we define six user personas including two non-expert residents (i.e., teachers) and two HVAC-expert building managers for the school site, and an additional pair (one resident and one manager) for the residential complex. Among them, non-expert residents are designed to query only their own HVAC units, using highly abstract and user-native language. On the other hand, expert building managers issue queries that focus on building-wide HVAC statistics, expressed in explicit

¹We have obtained the necessary permissions to analyze the data and publish results derived from both deployments.

Table 2. Database schema overview. The sensor data table contains 78 columns organized into five categories.

Category	Count	Representative Examples
Identifiers & Timestamps	6	id_site, id_indoorunit, server_time <i>Hierarchical unit IDs and temporal references</i>
Environmental Measurements	12	roomtemp, settemp, humidity <i>Temperature, humidity, and pipe sensor readings</i>
Operational States & Controls	28	oper, mode, fan, swing <i>Power status, mode, fan speed, and lock settings</i>
User Configurations	14	settemp_lim, cool_upper_lim <i>Setpoint limits, heating/cooling bounds, deadband</i>
Diagnostics & Availability	18	err, filter, occupied <i>Error codes, alerts, and feature availability flags</i>
Total	78	

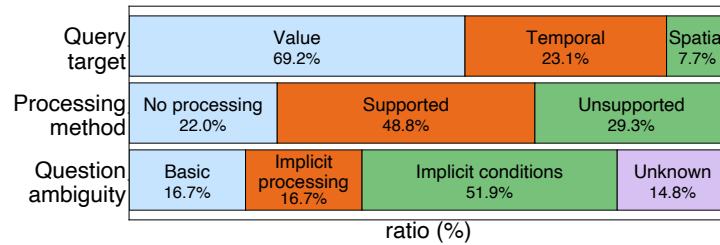


Fig. 6. Query type distribution for QA dataset.

and domain-specific terms. Each persona is associated with a distinct user-specific context (i.e., metadata), and queries span a range of time frames to reflect diverse informational needs.

6.1.2 QA Dataset Curation. To support model fine-tuning and use in our evaluations, we curated a dataset of 107 question-answer (QA) pairs spanning the six user personas and two real-world building sites. Each QA pair was authored by HVAC engineers with over two years of professional experience in developing industrial-scale HVAC control systems. These experts leveraged their domain knowledge and manually queried building sensor databases to write reference answers grounded in operational logic and real-world quality-of-experience (QoE) concerns. This ensured that the curated responses accurately reflect both system-level behaviors and end-user expectations. For data annotation, four core principles were followed: (i) implicit condition resolution, inferring defaults when queries omit spatial, temporal, or modality information based on user context; (ii) user expectation anticipation, including supplementary values users would implicitly want beyond the direct request; (iii) variable traceability, ensuring every placeholder in the Expectation template has a corresponding Script computation; and (iv) graceful failure handling, mapping unknown entities to “Unknown” and omitting the Script to enable informative failure messages rather than hallucinated responses. The QA pairs were then converted into system-specific formats for use with *JARVIS* and baseline models. In the case of *JARVIS*, each pair was decomposed into its full set of Expert-LLM components (e.g., *thinking*, *expectation*, and *instruction*) for detailed evaluation.

We allocated two of the four school-site personas (one teacher and one system manager) to the fine-tuning set, while all residential-site personas were reserved exclusively for the test set. This allows us to perform cross-site evaluation for *JARVIS* and validate its general applicability in various environments. The fine-tuning set includes

Table 3. Representative QA triples from our curated dataset. SQL queries include join operations, NULL handling, and NaN filtering consistent with real-world sensor data characteristics.

No.	Question	Metadata	SQL & Data Processing	Ground Truth Answer	Type / Notes
(1)	“What was the set temperature difference between the bedroom and living room yesterday?”	IDU_001: Bedroom IDU_002: Living room now: 2024-09-30 12:00	SELECT avg_bedroom, avg_livingroom, (avg_bedroom - avg_livingroom) AS settemp_diff FROM (SELECT AVG(CASE WHEN i.name = 'IDU_001' THEN d.settemp END) AS avg_bedroom, AVG(CASE WHEN i.name = 'IDU_002' THEN d.settemp END) AS avg_livingroom FROM sensor_data d JOIN idu_map i ON d.idu_id = i.id WHERE i.name IN ('IDU_001', 'IDU_002') AND d.timestamp >= '2024-09-29 00:00:00' AND d.timestamp < '2024-09-30 00:00:00' AND d.settemp IS NOT NULL AND d.settemp != 'NaN') AS aggregated_data;	“Yesterday, the bedroom’s average set temperature (27.00°C) was 4.00°C higher than the living room’s (23.00°C).”	Query target: value; implicit aggregation and multi-entity comparison
(2)	“When was the bedroom hottest recently?”	IDU_102: Bedroom now: 2024-09-30 08:00	SELECT TO_CHAR(d.timestamp, 'YYYY-MM-DD HH24:MI') AS date, AVG(d.roomtemp) AS avg_roomtemp FROM sensor_data d JOIN idu_map i ON d.idu_id = i.id WHERE i.name = 'IDU_102' AND d.timestamp >= '2024-09-23 08:00:00' AND d.timestamp < '2024-09-30 08:00:00' AND d.roomtemp IS NOT NULL AND d.roomtemp != 'NaN' GROUP BY TO_CHAR(d.timestamp, 'YYYY-MM-DD HH24:MI') ORDER BY avg_roomtemp DESC LIMIT 1;	“The bedroom was hottest on September 25th at 01:37 PM, reaching 35.00°C.”	Query target: temporal; argmax under implicit time window
(3)	“How many times was the AC turned on and off yesterday?”	IDU_201: Living room now: 2024-11-15 09:00	SELECT d.timestamp, d.oper FROM sensor_data d JOIN idu_map i ON d.idu_id = i.id WHERE i.name = 'IDU_201' AND d.timestamp >= '2024-11-14 00:00:00' AND d.timestamp < '2024-11-15 00:00:00' AND d.oper IS NOT NULL ORDER BY d.timestamp ASC; + <i>post-processing: count state transitions (ON↔OFF)</i>	“Yesterday, the AC was turned on 5 times and off 5 times.”	Query target: value; stateful processing required
(4)	“Which 4th floor units had overheating (>2°C) for >10 min this month?”	IDU_401-403: 4th floor now: 2024-08-20 14:00	SELECT i.name, d.timestamp, d.roomtemp, d.settemp FROM sensor_data d JOIN idu_map i ON d.idu_id = i.id WHERE i.name IN ('IDU_401', 'IDU_402', 'IDU_403') AND d.timestamp >= '2024-08-01 00:00:00' AND d.timestamp < '2024-08-20 14:00:00' AND d.mode = 'COOL' AND d.roomtemp IS NOT NULL AND d.settemp IS NOT NULL ORDER BY i.name, d.timestamp ASC; + <i>post-processing: filter $\Delta T > 2$, group consecutive, sum duration > 10min</i>	“Room 402: 45 min (3 episodes); Room 403: 28 min (2 episodes).”	Query target: spatial; multi-condition temporal aggregation
(5)	“What was the temperature on Mars?”	IDU_001-003: Bedroom, Living room, Kitchen	(None — entity not found in metadata)	“Sorry, ‘Mars’ is not a recognized space in the system.”	Unknown entity; graceful failure handling

34 and 19 QA pairs from the resident and manager personas (total 53), respectively, while the test set contains 17 and 10 QA pairs from the school site and another 17 and 10 from the residential site for cross-site evaluation. Note that the metadata used for fine-tuning and testing personas are completely disjoint, evaluating JARVIS under both zero-shot generalization (across user contexts) and cross-site generalization (between the school and residential deployments).

While the curated dataset of 53 training samples can be considered relatively small for fine-tuning purposes, this was primarily a deployment-driven decision, as collecting domain-specific HVAC query-response pairs

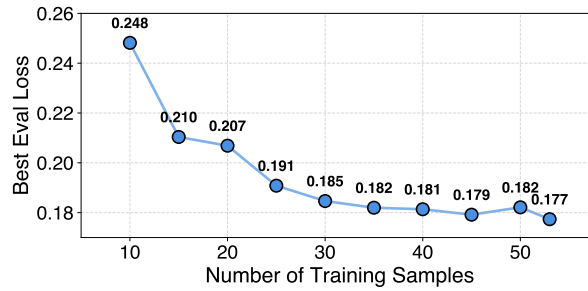


Fig. 7. Best-achieved evaluation loss per training session while fine-tuning the Expert-LLM with controlled number of training samples.

requires substantial expert involvement. Nevertheless, we provide an empirical justification for the dataset’s practical effectiveness. Specifically, Figure 7 plots the best-achieved evaluation loss as a function of the number of training samples. The loss decreases sharply in the early training stages and plateaus beyond approximately 40 samples, indicating diminishing returns from additional data. We attribute this training and data efficiency to our adaptive context injection strategy (Sec. 4.3). Specifically, in *JARVIS*, fine-tuning is applied exclusively to HVAC-common knowledge, which is deployment-agnostic and stable across scenarios within our target application. Dynamic information such as deployment-specific configurations, user preferences, and sensor data are instead injected via prompting at inference time. This separation significantly narrows the scope of what the model must learn through fine-tuning, reducing it to stable domain conventions and reasoning patterns rather than the full complexity of HVAC operations.

When examining detailed contents of our curated dataset, as Figure 6 shows, user queries are well-distributed along three dimensions of *question ambiguity*, *processing method*, and *query target*. A single query may belong to multiple categories within each dimension, as determined by the experts who created the dataset. For transparency and reproducibility, we adopt a multi-label tagging scheme where each query can carry multiple ambiguity tags, multiple processing operators, and a primary return type.

Table 3 presents five representative QA triple samples from the curated dataset. Each example includes a user query, the deployment-specific metadata available at query time, the data retrieval or post-processing logic required to compute the answer, the ground-truth response, and annotations describing the query type. These examples are selected to cover the major query categories shown in Figure 6 and to illustrate the range of reasoning and execution complexity that *JARVIS* must handle in practice. We note that the SQL expressions in Table 3 are included solely to expose the underlying complexity of the QA task, and are not used as supervision during training and testing. Instead, the Expert-LLM is fine-tuned to utilize the parameterized query execution interface, rather than generating raw SQL. This separation decouples semantic reasoning from database-specific syntax and allows the system to generalize across deployments with different schemas and implementations.

Question ambiguity captures how implicitly a query conveys its intended operation. Queries are labeled as involving *implicit processing* or *implicit conditions* when they omit explicit phrasing of data operations or constraints, especially when expressed using non-HVAC or colloquial terms. Here, *Unknown* queries refer to requests involving nonexistent data (e.g., invalid time ranges, missing modalities, or unmonitored spaces). In such cases, *JARVIS* is expected to return a truthful failure message rather than hallucinating an answer. This distribution demonstrates that the dataset includes diverse ambiguous and underspecified queries to test the Expert-LLM’s semantic grounding capabilities. For example, the second example in Table 3 shows a query where “recently” must be interpreted as an implicit but concrete time window (e.g., 7-days), and the fifth example presents a graceful failure handling case when the user requests data for a nonexistent location. To make ambiguity operational, we compose the dataset to let *JARVIS* normalize colloquial and implicit expressions into executable

scopes. For example, “yesterday” is normalized to a one-day interval (00:00-24:00), “this month” to the current calendar month, “last winter” to December through February, and “two weeks ago” to the calendar week two weeks prior. Spatial expressions such as “our classroom” or “4th floor” are resolved through a deployment-specific alias mapping.

Processing method indicates the level of data processing needed to answer the query. *No processing* denotes direct retrieval of stored values; *supported* refers to operations that can be handled using native SQL (e.g., filtering or aggregation); and *unsupported* denotes operations requiring additional computation (e.g., argmin, argmax) that must be executed in the Agent’s Python runtime. The third and fourth examples in Table 3 exemplify such *unsupported* cases. Specifically, in the example, counting AC on/off state transitions and computing consecutive overheating durations both require post-SQL processing in Python, as these operations involve stateful logic that cannot be efficiently expressed in standard SQL. This classification highlights the need for complex processing beyond standard database capabilities in typical HVAC applications.

Query target specifies the type of information requested. *Value* targets raw sensor readings, *temporal* consists of time-based queries (e.g., “when was...”), and *spatial* covers location-based queries, typically relevant to building managers. In Table 3, the first case is an example of a *value* query comparing set temperatures across two rooms, the second case presents a *temporal* query identifying when a room was hottest, and the fourth example presents a case where a *spatial* query aggregates overheating statistics across multiple units on the same floor. Such diversity in the dataset allows comprehensive evaluation of the system’s ability to handle a range of reasoning and retrieval tasks.

6.2 Evaluation Metrics

Evaluating the performance of LLM-based systems on real-world data is inherently challenging due to the limited availability of large-scale labeled datasets and the subjective nature of natural language interpretation. To address this, we define a set of both quantitative and qualitative metrics, including relevant metrics adopted from the open-source RAG evaluation framework RAGAS [11], to assess the performance of JARVIS. These metrics are chosen to capture different aspects of data querying accuracy and response quality, as detailed below.

Data querying accuracy metrics. The accuracy of data query operations are evaluated using three widely-used SQL-based-retrieval metrics [11, 66]. These metrics evaluate the accuracy of the SQL statements generated by JARVIS, which involves successful operations at the Expert-LLM and the Query Execution module at the Agent..

- **Execution Accuracy** [66]: True if the queried data exactly matches the ground truth query, else false.
- **Precision** [11]: Proportion of queried data that are actually correct.
- **Recall** [11]: Proportion of correct data among all queried data.

We note that in reporting these values, we aggregate each metric by taking the average values per query.

Response quality metrics. We evaluate the quality of final responses generated by JARVIS using two complementary classes of metrics: (i) judgment-based metrics, providing holistic, human-aligned assessments of response quality, and (ii) reference-aware metrics, which explicitly quantify alignment to external references such as the user query or the retrieved context.

Following prior work [8, 26, 39, 59, 68], we adopt three judgment-based metrics that assess different dimensions of response quality. Each metric is rated on a 5-point Likert scale:

- **Question-Answer Cohesiveness** [26, 39, 59, 68] measures the semantic alignment between the user’s question and the generated answer (i.e., how closely the content of the answer pertains to the subject and intent of the question).

- **Helpfulness** [26, 39, 59, 68] assesses the extent to which the answer delivers clear and self-contained information that requires minimal user interpretation or reasoning, as well as the presence of relevant auxiliary information that, while not explicitly requested, may enhance user understanding or utility.
- **Truthfulness** [8, 26, 39, 68] evaluates the factual accuracy of the response compared to a given ground truth, particularly for verifiable elements such as numerical values, factual claims, or explanatory statements.

Throughout the evaluation, excluding the user study, we assess response quality using the *LLM-as-a-Judge* methodology [24, 26, 49, 57, 72]. In this approach, external LLMs serve as surrogate evaluators. Specifically, we employ GPT-4o, GPT-3.5-turbo, and Gemini-2.5-pro, which are state-of-the-art models in reasoning tasks. Each model evaluates the responses independently in two separate runs, resulting in a total of six scores per response. We report the average of the three metrics as the final score unless analyzed individually. This evaluation strategy enables large-scale, consistent assessment of qualitative response quality, bypassing the need for labor-intensive human annotation.

In addition to judgment-based evaluation, we incorporate two reference-aware metrics from the RAGAS framework [11]. These metrics explicitly condition on external references (the user query and/or retrieved context) and yield scores between 0 and 1.

- **Faithfulness** [11] measures whether factual statements in the generated response are supported by the retrieved context. RAGAS decomposes the response into atomic claims and checks whether each claim can be entailed by the provided context. The final score reflects the proportion of claims grounded in the retrieved evidence, penalizing unsupported or hallucinated statements.
- **Answer Relevancy** [11] measures how well the generated answer aligns with the original user query in terms of relevance and coverage. RAGAS estimates relevance by estimating three implicit questions from the answer and comparing these generated questions to the original query using LLM-assisted vectorization and cosine similarity. Scores decrease when the response is generic, partially addresses the query, or deviates from the intended question.

Similar to the judgment-based evaluation, we use GPT-4o as the evaluation LLM for computing RAGAS metrics.

System performance metrics. To evaluate the efficiency of *JARVIS*, we measure end-to-end latency, defined as the time elapsed from receiving the user’s input to the generation of the final response token. All system performance evaluations are conducted on a desktop-level setup with two NVIDIA RTX 4080 Super GPUs (16 GB VRAM each). For evaluations involving the 70B model we use the NVIDIA H100 GPU (80 GB VRAM) to overcome the memory limitations of the RTX 4080 GPU. Additionally, we report the total token length, calculated by summing the output token length of the front-end model (i.e., the Expert-LLM in *JARVIS*) and both the input and output token lengths of the back-end model (i.e., the response-generation LLM). This total includes the system prompt but excludes any padding or internal tokens used purely for generation. Token length is a critical efficiency metric, as it directly affects inference latency and computational cost, with longer input/output sequences increasing generation time and system resource usage.

6.3 End-to-end Response Quality Performance

We start by evaluating the end-to-end response quality of *JARVIS* to assess its overall system-level effectiveness using the five response quality metrics introduced above. To support a more comprehensive evaluation on a larger test set, we adopt an LLM-as-a-Judge methodology as the primary evaluation approach. This choice is motivated by the need for scalable, consistent, and fine-grained assessment across a large number of queries, which would be impractical to achieve with human evaluations while maintaining evaluation consistency. Figure 8 summarizes the comparative performance of *JARVIS* against two complementary baseline categories that reflect plausible design choices for LLM-based, data-driven question answering systems. We include both baselines to contextualize performance across unstructured agentic reasoning and structured database-oriented pipelines.

The first baseline category is an end-to-end agentic baseline, in which an off-the-shelf LLM handles the entire pipeline within a shared conversational context. The LLM in this case is informed that database access (via SQL) is available, but is not explicitly instructed to generate queries; instead, it autonomously decides whether and how to leverage the database during response generation. This baseline represents a natural and commonly attempted first approach, requiring minimal system engineering while fully leveraging the reasoning capabilities of powerful off-the-shelf models. The second baseline is TAG [5], a structured RAG framework for tabular databases that explicitly enforces SQL-based data access prior to response generation. Unlike the agentic baseline, TAG, similar to the *JARVIS* architecture, instructs its LLM to generate SQL queries through explicit prompting, after which a separate, smaller local LLM (LG Exaone-3.5-7.8B, matched to *JARVIS* for fairness) produces the final response without access to the query formulation context. This design closely resembles *JARVIS* in its use of structured data access, while differing in LLM architectural choices and the incorporation of HVAC-specific domain knowledge.

Note that we include both baselines for complementary reasons. The agentic baseline captures an intuitive, low-engineering entry point that many practitioners would naturally adopt. TAG, on the other hand, serves as our primary structured baseline, representing the closest existing framework for bridging natural language queries and structured databases through explicit stage separation. Given the lack of prior work on LLM-based RAG frameworks tailored to sensor-driven HVAC question answering, TAG provides the most relevant point of comparison for evaluating HVAC-aware system design.

All baselines are provided with the same basic contextual information describing the deployment environment, task scope, and available database schema to ensure a fair comparison. We intentionally select mid-tier off-the-shelf models (e.g., Gemini-3-flash, Qwen3-VL-32B, GPT-5-mini) rather than the largest state-of-the-art alternatives to ensure cost fairness relative to *JARVIS*'s local 8B Expert-LLM. For these agentic baselines, we report results under zero-shot prompting only, as the model autonomously determines its reasoning strategy without explicit guidance. For the TAG baseline, we report results under both zero-shot and three-shot prompting settings. Local LLMs, including the Text2SQL variant of TAG and the Expert-LLM in *JARVIS*, were also evaluated using fine-tuned configurations using on the 53 HVAC question-answer pairs to examine the effect of lightweight domain adaptation beyond in-context prompting. Finally, in evaluating *JARVIS*, we evaluate three-shot and fine-tuned Expert-LLM configurations using the same end-to-end framework. We do not report a zero-shot setting for *JARVIS*, as such a configuration is not representative of the intended system design.

Overall, the results in Figure 8 show that the fine-tuned configuration of *JARVIS* achieves competitive or superior performance across most response quality metrics, outperforming both agentic and TAG baselines in three of the five evaluation dimensions (i.e., cohesiveness, truthfulness, and faithfulness). Based on the reasoning traces produced by the LLM judge and a qualitative inspection of representative samples, we attribute these gains to two key design factors that directly reflect our core design goals. First, semantic context injection at the Expert-LLM enables *JARVIS* to more accurately interpret user intent in HVAC-specific terms (**G1; c.f., Sec. 4.1**), resulting in responses that are better aligned with the underlying query semantics. Second, the explicit integration of structured data processing through the Data Processing module improves factual accuracy by offloading statistical computation and aggregation from the language model (**G2**). This design reduces reliance on implicit arithmetic reasoning within LLMs and directly contributes to higher truthfulness and faithfulness scores.

The performance gap can be better explained when individually comparing *JARVIS* against each baseline category. Agentic baselines rely on a single LLM to autonomously handle query formulation and response generation within a shared context. While this design leverages the model's reasoning capabilities, it places the full burden of SQL generation, data interpretation, and response synthesis on one model without domain-specific guidance, increasing susceptibility to cascading errors. On the other hand, TAG addresses this by explicitly separating SQL generation from response generation and by using a dedicated local LLM for the latter. However, both approaches share two common limitations: (1) the absence of an expert-level front-end that calibrates

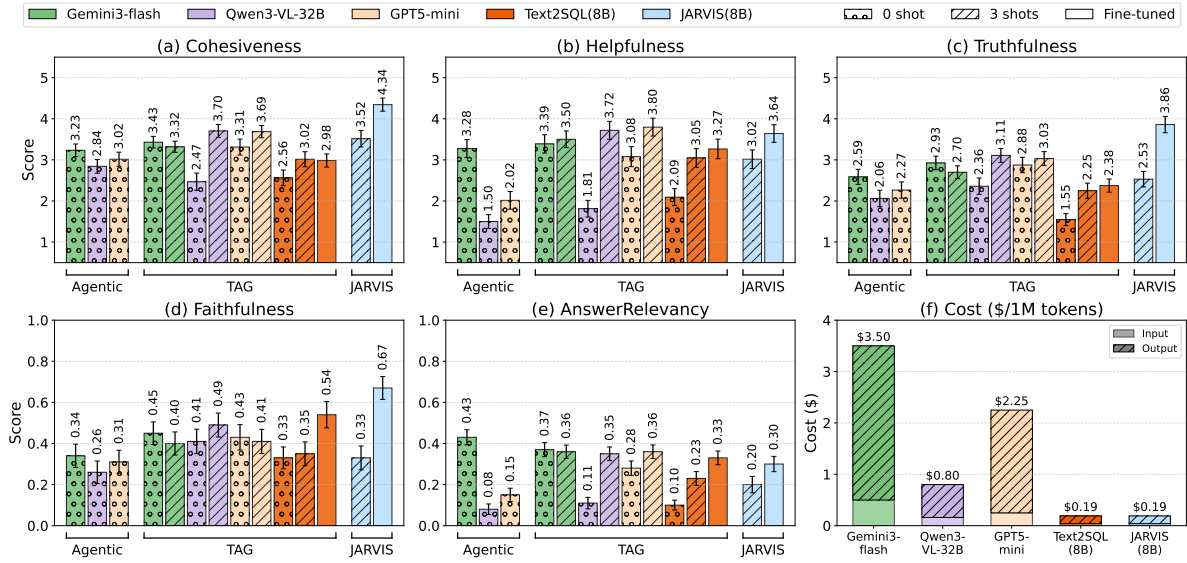


Fig. 8. (a-e): Response quality of *JARVIS* vs. families of baselines (Agentic, TAG). Higher the better; (f): Cost comparison between off-the-shelf LLMs (Gemini-3-flash, Qwen3-VL-32B, GPT-5-mini) and local LLMs (LLaMA 3 8B families, fine-tuned for Text2SQL and *JARVIS*). Lower the better.

user inputs using HVAC-specific semantic context makes them susceptible to misinterpreting vague or domain-specific expressions, and (2) their reliance on the LLM to reason directly over raw query results leads to increase in likelihood of hallucination or can result in inconsistent aggregation when handling large or complex data. Furthermore, in several cases, both agentic and TAG baselines successfully retrieved relevant data but failed to produce a response because the retrieved records exceeded the input context limit of the response-generation LLM. *JARVIS* addresses these limitations through explicit multi-stage decomposition (G3) with the Expert-LLM handling query interpretation with HVAC-specific context, the Data Processing module performing aggregation, and a separate response generator synthesizing the final answer. This separation ensures coherence and goal alignment across stages, avoiding the cascading errors observed in agentic baselines; thus, yielding the largest margin in truthfulness, where *JARVIS* achieves 3.86 compared to 3.11 for the best TAG baseline (Qwen3, 3-shot).

From a cost-efficiency perspective, *JARVIS* demonstrates a favorable trade-off that supports practical deployment (G4). As plotted in Figure 8 (f), *JARVIS* operates at approximately \$0.19 per million tokens, which is about 10× lower than exploiting a GPT-5-mini (\$2.25) and nearly 18× lower than Gemini-3-flash (\$3.50)². Despite relying on a relatively small local Expert-LLM (practically not requiring additional token costs), *JARVIS* achieves response quality comparable to, or exceeding, that of agentic baselines built on substantially larger off-the-shelf models, highlighting its efficiency-performance advantage for real-world deployment.

Nevertheless, we observe limitations in two metrics. In terms of helpfulness, *JARVIS* achieves competitive but non-leading scores (3.64), with TAG using GPT-5-mini performing higher (3.80 in the three-shot setting). More notably, answer relevancy scores for *JARVIS* are consistently lower than those of all baselines. Qualitative analysis of failure cases reveals two contributing factors. First, in edge cases where numerical outcomes deviate from user expectations (e.g., comparing identical values), *JARVIS* produces rigid, literal responses (e.g., “larger by 0”) rather than more natural formulations (e.g., “the same”), reducing perceived relevance. Second, when

²Token pricing for GPT and Gemini is taken from OpenAI and Google, respectively, while pricing for Qwen and LLaMA (including *JARVIS*) is based on together.ai.

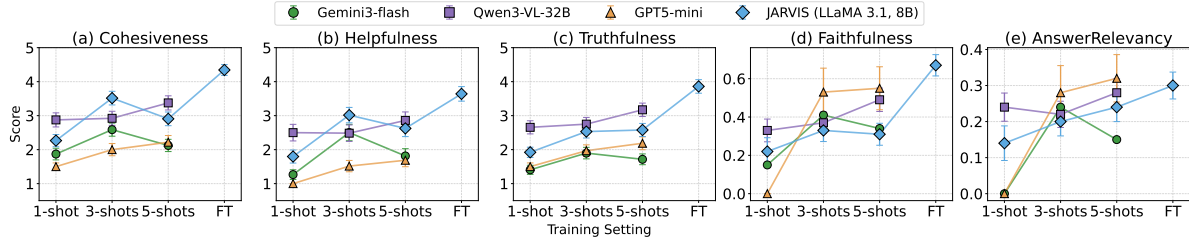


Fig. 9. Response quality comparisons for different Expert-LLM base models under 1-shot, 3-shot, 5-shot prompting and fine-tuning. Higher the better.

queries involve nonexistent modalities, spaces, or unavailable data, *JARVIS* prioritizes factual correctness by explicitly reporting failure but provides limited explanation of which part of the query is invalid or missing. For instance, when a user asks “What was the current temperature of Mars last week?”, *JARVIS* responds with “Sorry, ‘Mars’ is not a recognized entity in the system,” rather than merely repeating that the requested information is unavailable (e.g., “The temperature of ‘Mars’ last week cannot be retrieved because it is not a valid entity in the deployment.”). While responses remain factually grounded and do not significantly degrade cohesiveness, given that the failure is inherent to the original query and correctly identifying this failure is itself a coherent outcome, they often require non-intuitive user inference to fully understand. This additional cognitive burden reduces perceived helpfulness and contributes to lower Answer Relevancy scores under metrics that emphasize direct question-answer alignment, such as RAGAS.

These observations highlight an inherent trade-off in *JARVIS*’s current design, which is the fact that the system prioritizes factual correctness and faithful grounding over conversational flexibility and error explanation. While this design choice improves truthfulness and faithfulness, it can lead to less intuitive responses in edge cases, suggesting opportunities for future improvements in adaptive response phrasing and user-facing error feedback. Despite these limitations, our results suggests that *JARVIS* achieves strong performance across the majority of metrics while operating at substantially lower cost than comparable baselines.

One point to emphasize is that these gains are observed generally across different model choices and adaptation strategies, indicating that the improvements stem from the principled system-level design of *JARVIS* rather than from specific prompt engineering or fine-tuning alone. Overall, the results validate that explicitly structuring intent interpretation (G1), data retrieval (G2), and coherent multi-stage reasoning (G3) yields a robust and cost-effective strategy (G4) for reliable, sensor-driven HVAC question answering.

6.4 Base Model for the Expert-LLM

Prior to presenting component-level analysis, we first validate our key design choices for the Expert-LLM, a core component in *JARVIS*. Specifically, *JARVIS* employs a fine-tuned LLaMA3.1-8B-Instruct model. We evaluate the Expert-LLM to answer the following questions: (i) “Why fine-tune a local model when powerful off-the-shelf SOTA models are readily available via API?” and (ii) “Among local models, why choose the 8B variant over smaller or larger alternatives?” These questions validate G4 and we target to provide empirical justification for these decisions in this section.

6.4.1 Off-the-shelf LLMs with Prompting vs. Fine-tuned Local LLM. Given the strong reasoning capabilities of modern SOTA LLMs such as GPT-5 and Gemini-3, when disregarding the need for continuous connectivity, one might question whether fine-tuning a smaller local model is necessary. An alternative approach would be to leverage these powerful models directly as the Expert-LLM via in-context learning. To investigate whether model fine-tuning provides meaningful benefits over this approach, we compare *JARVIS*’s fine-tuned LLaMA3.1-8B against Gemini-3-flash, Qwen3-VL-32B, and GPT-5-mini serving as the Expert-LLM under 1-shot, 3-shot, and

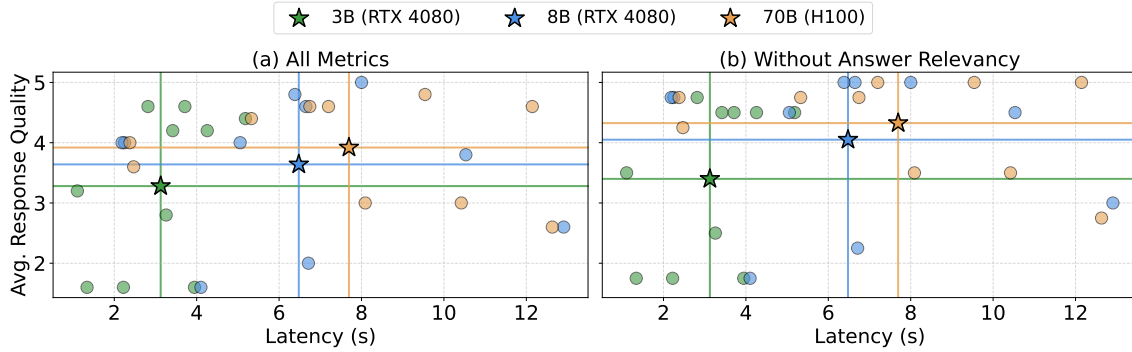


Fig. 10. Average response quality vs. Response latency for different Expert-LLMs of varying parameter sizes. Experiments for the 3B and 8B models were done using the RTX 4080 GPU while 70B model experiments were done on the H100 GPU.

5-shot configurations. Note that fine-tuning these baseline LLMs are not practical when using them off-the-shelf from the cloud.

As Figure 9 plots, fine-tuned *JARVIS* consistently outperforms all SOTA models across judgment-based metrics (cohesiveness, helpfulness, truthfulness) and faithfulness. While SOTA models show gradual improvements as the number of in-context examples increases from 1-shot to 3-shot, such approaches introduce practical drawbacks. First, increasing the number of examples proportionally will increase token length, leading to higher inference costs. As previously discussed using Figure 8 (f), SOTA models already operate at 10-18 $\times$ higher cost per token compared to local deployment. Second, even with 5-shot, SOTA models do not reach the performance level achieved by fine-tuned *JARVIS*, and some models even exhibit performance degradation for certain metrics. In contrast, fine-tuning provides a substantial and stable performance gain without incurring additional per-query costs, particularly in cohesiveness (2.9 to 4.3) and faithfulness (0.4 to 0.65).

We attribute this performance advantage to the nature of HVAC-common knowledge required for the Expert-LLM role. As discussed in Section 4.3, HVAC-common knowledge encompasses domain-specific reasoning patterns, linguistic conventions in user queries, and presuppositions underlying HVAC-related requests. This type of context is deployment-agnostic, but requires lengthy descriptions and diverse examples to convey effectively. Such characteristics make fine-tuning the optimal injection method, since once embedded into model weights, the knowledge remains robust and consistently accessible during inference without increasing prompt length or inference cost.

These results validate our design choice to fine-tune a local Expert-LLM, suggesting that it provides better performance at lower operational cost compared to relying on SOTA cloud-based LLMs with in-context learning.

6.4.2 LLM Size Selection. Given the decision to fine-tune a local model, we next investigate which model size best balances response quality and latency. We compare three models from the LLaMA 3 family: 3B (LLaMA-3.2-3B), 8B (LLaMA-3.1-8B-Instruct), and 70B (LLaMA-3-70B). Each model was fine-tuned on the same curated subset of HVAC QA samples and evaluated on representative queries spanning the full range of user intents described in Figure 6. Due to memory limitations, the 70B model was executed on an NVIDIA H100 GPU, whereas the 3B and 8B models used two RTX 4080 Super GPUs. All implementations were optimized using NVIDIA TensorRT-LLM.

Figure 10 (a) presents the trade-off between average response quality (across all five metrics) and end-to-end latency for each model. We note that all configurations exhibit some low-scoring responses, largely due to the Answer Relevancy metric, which tends to penalize *JARVIS*'s prioritization of factual correctness over conversational fluency (discussed in Sec. 6.3). When excluding Answer Relevancy and averaging only the remaining four metrics, we observe a noticeable increase in overall response quality scores, as shown in Figure 10 (b).

Based on the plots, the overall trends are as follows. The 70B model, as expected, achieves the highest average response quality (3.92 across all five metrics and 4.33 when excluding Answer Relevancy). However, we note that its latency measurements were obtained using an NVIDIA H100 GPU, which is not practical for typical on-site deployments. Without access to such high-end infrastructure, we anticipate significantly higher latency, rendering the 70B model unsuitable for real-time HVAC QA applications. In contrast, the 3B model exhibits substantially lower latency (average: 3.1 seconds), but its response quality is highly inconsistent. Even after excluding the Answer Relevancy metric, three out of ten responses scored below 2. Further analysis revealed that these low-quality outputs were nearly impossible to process downstream, due to either incoherence or factual inaccuracy. This suggests that the 3B model lacks sufficient capacity for reliable reasoning in the HVAC domain.

The 8B model offers a more favorable balance between response accuracy and latency; achieving an average response quality of 3.64 across all five metrics and 4.05 when excluding Answer Relevancy, with a latency of 6.4 seconds, approximately double that of the 3B model, but still within acceptable limits for interactive QA scenarios. Given the importance of response quality in our application context, and the feasibility of on-device or on-site deployment with this latency profile, we adopt the 8B model as the default Expert-LLM configuration for JARVIS. Nevertheless, we note that JARVIS is designed to be modular and allows practitioners to substitute alternative models based on deployment-specific constraints such as target language, latency requirements, or hardware availability, without modifying the overall system architecture.

6.5 Component Level Response Quality

Given the end-to-end performance of JARVIS, we now shift focus to a more fine-grained, component-level analysis of response quality. The goal of this analysis is to assess the effectiveness of each core design choice in JARVIS and understand how individual components contribute to overall system performance, thereby validating the design goals outlined in Section 4.1. To this end, we compare JARVIS against several ablation variants, where one or more key modules are removed. This comparison highlights the benefits of JARVIS's architecture and quantifies the contribution of each component to average response quality.

We evaluate seven configurations in total. First, we remove both the Metadata Injection and the Thinking Module from the Expert-LLM (w/o M&T), simulating a generic model with no domain adaptation or tailored reasoning capabilities. This configuration directly show JARVIS satisfies **G1** (user-native query interpretation) and **G3** (coherent multi-stage reasoning). Second, we reintroduce Metadata while omitting the Thinking Module (w/o Thinking), reflecting a configuration with contextual data but no specialized reasoning. Third, we test a setup with Metadata and Thinking enabled, but with the Expectation module removed (w/o Expect.), which prevents the system from aligning instructions with inferred user goals, to validate if **G3** (coherent multi-stage reasoning through goal-conditioned planning) is satisfied.

Turning to the Agent component, we remove the parameterized Query Execution module (w/o QueryExec.), responsible for translating instructions into robust SQL queries. We then remove the Data Processing module (w/o Processing), assessing performance when statistical operations rely solely on native SQL. We also examine a configuration with both of these modules removed (w/o Q&P), resulting in a minimal backend. These configurations assess how JARVIS addresses **G2** (precise and contextual retrieval) and **G4** (robust deployment). All variants are compared against the full implementation of JARVIS.

6.5.1 Response Quality over Query Execution Accuracy. As Figure 11 shows, JARVIS, equipped with all components, consistently outperforms its ablation variants in response quality. Notice that while query accuracy and response quality are generally well-aligned, we see some notable exceptions. For instance, the w/o Q&P configuration yields comparable (and occasionally higher) query match accuracy compared to the overall linear trend, but suffers from markedly lower response quality. Similarly, while both w/o QueryExec and w/o Processing achieve comparable levels of query execution accuracy, w/o QueryExec consistently delivers higher response quality. This



Fig. 11. Average response quality vs. Query execution accuracy. Better toward top-right.

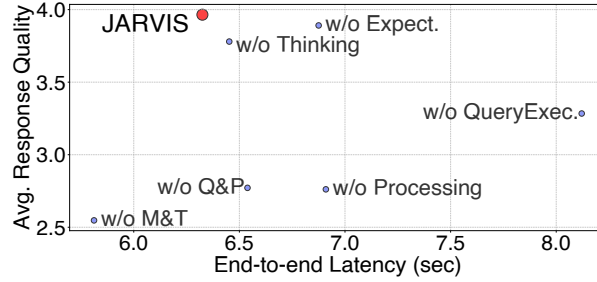


Fig. 12. Avg. response quality vs. End-to-end latency. Better toward top-left. The end-to-end latency are measured using RTX 4080 GPUs.

indicates that accurate retrieval alone cannot guarantee effective reasoning, and emphasizes the role of statistical processing in transforming retrieved data into meaningful, high-quality answers. The improvement gained by adding the Processing module aligns with our preliminary findings in Section 3.1, confirming that algorithmic post-processing enhances LLMs’ ability to handle long data streams and produce coherent, numerically faithful responses. Consequently, the usefulness of the Processing component is further reinforced as it ensures higher response quality from the same queried results.

We further observe a super-additive synergy when both the Query Execution and Processing modules are used together. Ablating either component degrades the query execution accuracy, while combining both (c.f., full *JARVIS*) achieves notable improvement. Further analysis reveals that the Query Execution module produces deterministic SQL queries with predictable result schemas, while the Processing module generates Python logic tailored to well-consume the schemas. This alignment reduces execution errors and boosts end-to-end robustness.

A similar pattern holds for w/o M&T, w/o Thinking, w/o Expectation, and the full *JARVIS* configuration. Metadata injection (compare *JARVIS* with w/o Thinking) improves both query accuracy and response quality by enriching user intent with contextual information, validating that *JARVIS*’ design satisfies **G1**. Adding the Thinking component further enhances performance by enabling structured reasoning over ambiguous or underspecified queries. Moreover, introducing the Expectation component at the beginning of the reasoning process (c.f., the bottom-up planning in Section 4.4.5) ensures that *JARVIS* proactively identifies and queries all data necessary to fulfill the user’s request, confirming the importance of a multi-stage reasoning process (**G3**). Together, these results confirm the architectural value of semantic grounding, intermediate reasoning, and goal-conditioned planning in achieving accurate and coherent multi-stage responses.

6.5.2 Response Quality over End-to-end Latency. Figure 12 presents the trade-off between response quality and end-to-end inference time. Interestingly, *JARVIS*, which achieves the highest response quality, also ranks as the second-fastest configuration among all variants. While one might expect that omitting the Thinking component

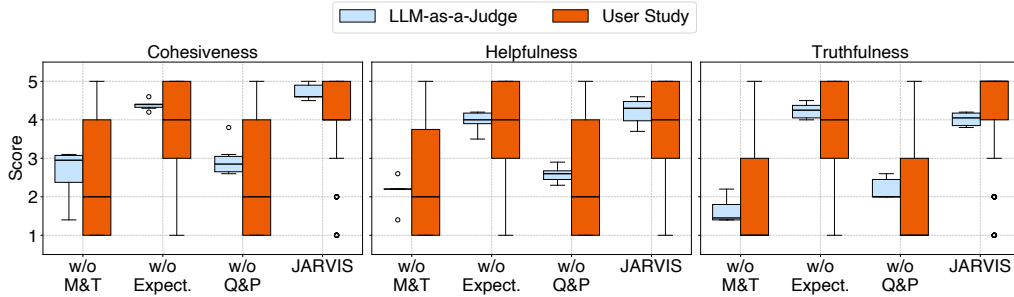


Fig. 13. Box plot of response quality measure comparison between the LLM-as-a-Judge approach and user study (higher is better). Error bars indicate 25th–75th percentile range and horizontal lines in each box represent the median.

(i.e., w/o M&T and w/o Thinking) would reduce inference time due to fewer tokens being generated, we find a counter-intuitive result with the full *JARVIS* that includes the Thinking component being slightly faster than w/o Thinking. This is because the Thinking component enables more optimized execution paths by planning efficient queries and processing steps, which compensates for the additional token cost during generation. A similar benefit is observed with the Expectation module, which helps guide the system toward efficient and focused responses.

On the other hand, the slowest configuration was w/o QueryExec. Without the parameterized Query Execution module, the Expert-LLM must generate complete SQL statements, including all boilerplate syntax and joins, which substantially increases both token length and reasoning overhead. Note that w/o Q&P shows comparably lower latency even without the Query Execution module because it skips generating and executing processing scripts, contributing to nontrivial delay. Overall, these results demonstrate that *JARVIS* is a performance-efficient framework that achieves both high response quality and low inference latency through principled modular design, fulfilling the requirements for **G4** (efficient, adaptable, and robust deployment).

6.6 Response Quality User Study

While the LLM-as-a-Judge framework offers a scalable and cost-effective means of evaluating model outputs, it remains an automated proxy. To complement this, we conducted a user study aimed at (i) validating the reliability of automated scores, and (ii) capturing qualitative aspects of user experience that may be overlooked by automated metrics.

We conducted an IRB-approved user study using a curated subset of the test set. The study evaluated 10 representative *user query–system response* pairs, each drawn from different *JARVIS* configurations. Participants rated each response on a 1–5 Likert scale across three quality dimensions: cohesiveness, helpfulness, and truthfulness. To ensure consistent evaluation, each participant was provided with a brief scenario description and a calibration example designed to establish the intended persona and scoring rubric. Note that the 10 selected questions covered the entire range of query types in our dataset described in Figure 6.

The study involved 22 participants (mean age: 35.9, SD: 11.0). Among them, 45.7% reported daily HVAC system usage and expressed a good understanding of its operation, while 8.6% identified as professionals working in HVAC-related industries. Regarding familiarity with LLMs, 45.5% reported moderate experience, and 24.2% stated that they primarily use LLMs for question answering and information retrieval tasks.

6.6.1 Overall Analysis. Figure 13 compares the average response quality scores assigned by the LLM-as-a-Judge framework with those obtained from the user study for the same 10 data points. While user ratings exhibit higher variance, the LLM-based scores closely track the median of the human evaluations. Furthermore, the relative ranking of the ablation variants remains highly consistent across both evaluation methods, suggesting that the

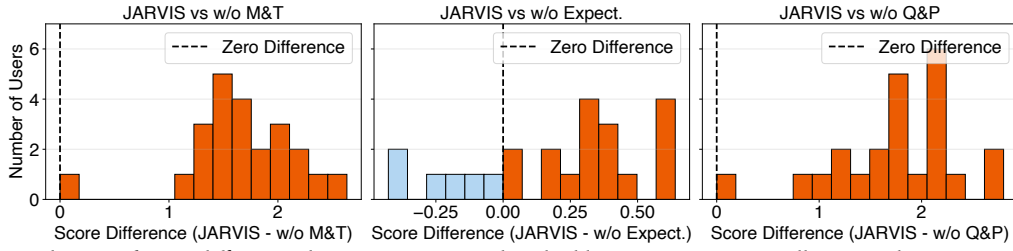


Fig. 14. Distribution of score differences between *JARVIS* and each ablation variant across all user study participants. Orange bars indicate users preferring *JARVIS* while blue bars indicate users preferring comparisons.

automated LLM-as-a-Judge serves as a reliable proxy for human assessment within our dataset and application scenario. As expected, the full implementation of *JARVIS* achieves the highest scores across all three response quality dimensions, indicating strong overall performance and consistent user preference across diverse query types.

To further assess the statistical significance of these findings, we independently analyzed the user study results. Figure 14 shows the distribution of per-user score differences between *JARVIS* and three ablation variants, across all participants and response metrics. The histograms reveal that *JARVIS* consistently achieves higher scores, with most differences falling on the positive side. In particular, *JARVIS* outperforms all participants when compared to w/o M&T and w/o Q&P, and still demonstrates a positive trend over w/o Expect., albeit with a few cases where users slightly preferred the ablation’s output. Overall, the distributions are right-skewed, reflecting a meaningful margin of improvement.

Since each participant rated all configurations on the same set of questions, we treated the samples as paired and performed within-subject comparisons across system variants. Due to visible non-normality in the score differences, we applied the Wilcoxon signed-rank test to assess whether *JARVIS* achieved significantly higher median scores. The results indicate that median differences were statistically significant: $p = .001$ for w/o Expect., and $p < .0005$ for both w/o Q&P and w/o M&T. These results confirm that *JARVIS* significantly outperforms all ablations in user-rated response quality.

Moreover, the corresponding effect sizes were large ($r = 0.64, 0.86, \text{ and } 0.88$, respectively), highlighting strong practical significance. Taken together, these findings demonstrate that *JARVIS* produces responses that are not only statistically better but also meaningfully more cohesive, helpful, and truthful from a user experience perspective.

6.6.2 Per-Question Analysis. We further analyze the user study on a per-question basis to gain deeper insights into how specific queries are handled across different system configurations. Question 1 posed the following query from a school HVAC manager persona while inspecting a specific classroom: “What is this week’s average temperature of this classroom and the classroom next door?” Participants evaluated responses from all system variants using the three response quality metrics. As shown in the leftmost plot in Figure 15, *JARVIS* is the only configuration that consistently receives high scores across all metrics.

A key observation is the poor performance of the w/o M&T variant, which fails to generate valid SQL due to ambiguous references such as “this classroom” and “the classroom next door.” In contrast, *JARVIS* leverages meta-data injection and the semantic reasoning capabilities of the Thinking module to disambiguate such expressions, enabling the Expert-LLM to produce context-aware and deployment-specific queries.

In the w/o Q&P variant, the system correctly identifies the user’s intent and generates a plausible answering plan. However, the absence of statistical processing results in inaccurate computations, leading to a significant drop in factual correctness (i.e., Truthfulness). This highlights the critical role of the Agent’s Query Execution and Processing modules in ensuring mathematically sound and trustworthy responses.

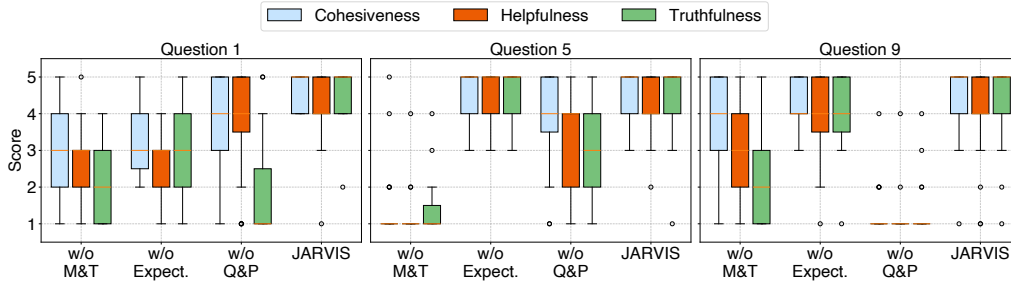


Fig. 15. Box plot of user study results for Questions 1, 5 and 9. Error bars indicate 25th–75th percentile range and horizontal lines in each box represent the median.

Interestingly, in the w/o Expect case, the Expert-LLM correctly interprets the user’s request and outlines a reasonable high-level plan. Yet, its execution instructions deviate from the intended logic—combining temperature data from both rooms into a single average instead of reporting them separately. This misalignment results in reduced cohesiveness and helpfulness, as the response fails to meet the user’s implicit expectations. In contrast, full JARVIS leverages the Expectation module to define a clear reference output format (e.g., two distinct values), guiding the instruction generation process. This illustrates an instance of our *bottom-up planning* design, where the system first establishes a response goal and uses it to shape downstream reasoning.

In Question 5 (center plot in Figure 15), a non-expert teacher (school resident) persona asks: “When was the hottest day two weeks ago?”. This query involves implicit reasoning and vague conditions. The system must infer that “hottest” refers to the peak indoor temperature and that the lack of spatial reference implies the teacher’s classroom. Such inferences rely on HVAC-domain common sense. The w/o M&T variant underperforms here, lacking the contextual reasoning needed to interpret domain-specific terminology. The w/o Q&P variant also performs poorly, as identifying the “hottest” day requires an argmax operation, which is unsupported by SQL alone and requires additional logic in the processing module. Without it, the system fails to retrieve the correct result, leading to a substantial drop in truthfulness.

In Question 9, a non-expert building resident asks: “What was the highest and lowest room temperature in the last summer?”. This query tests the system’s ability to resolve ambiguous temporal expressions; specifically, interpreting “summer” as a concrete date range. General-purpose LLMs often struggle with such under-specified conditions. In the w/o M&T configuration, the system grasps the intent but fails to map “summer” into a usable temporal constraint. This reinforces the importance of the Thinking module, which grounds vague expressions in HVAC-specific contexts.

Moreover, in the w/o Q&P configuration, the system is unable to convert the temporal filter into executable SQL due to the lack of a structured query module. Even if the temporal window is inferred correctly, the system cannot act on it; further emphasizing the importance of back-end processing components for domain-specific reasoning.

6.7 System-level Query Accuracy Analysis

As briefly discussed in Section 6.5, query accuracy tends to show a positive correlation with the overall response quality. In this section, we conduct a deeper analysis of how each component in JARVIS affects data querying performance by decomposing the query accuracy metric into precision and recall.

With the full JARVIS showing the best overall performance, Figure 16 provides a deeper view of the synergy between the Query Execution and Processing modules, as previously observed when analyzing Figure 11. Compared to w/o Q&P, both w/o QueryExec and w/o Processing exhibit clear performance degradation, though for different reasons. In the w/o Processing case, precision remains slightly higher, but recall drops sharply.

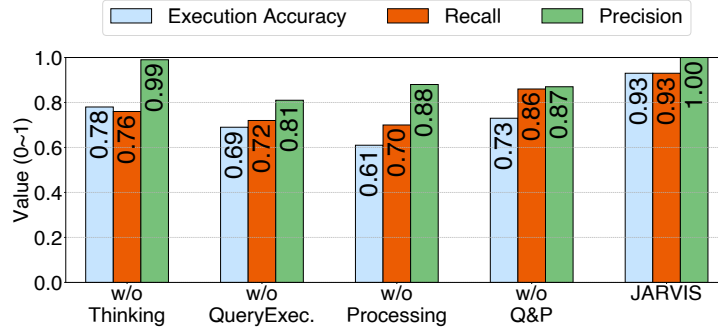


Fig. 16. Query execution accuracy, recall and precision for different *JARVIS* variants. Higher the better.

Latency (sec)	Expert LLM	Querying	Processing	Response
Proc. X	4.31 ± 0.68	0.00 ± 0.00	0.24 ± 0.01	0.39 ± 0.07
Proc. O	6.94 ± 3.74	0.00 ± 0.01	1.75 ± 2.56	0.56 ± 0.35
Ambig. X	4.82 ± 1.98	0.01 ± 0.02	0.59 ± 0.78	0.55 ± 0.49
Ambig. O	7.16 ± 3.51	0.00 ± 0.00	1.85 ± 2.67	0.53 ± 0.31

Table 4. Latency observed at each processing module within *JARVIS* for different types of input query requests. The latencies are measured using RTX 4080s.

This indicates overly conservative retrieval, often caused by incomplete SQL responses when post-query logic is absent, a limitation which *JARVIS* addresses by satisfying **G2**. For example, when handling temporal aggregation requests such as “When was the first day the HVAC unit was turned on?”, the system tends to return only the first available record because the raw SQL layer cannot infer the required temporal semantics, resulting in partial or truncated answers.

In contrast, w/o QueryExec shows a bit milder decline in recall but a more noticeable drop in precision, suggesting that the query execution module acts as a stricter filtering mechanism. Without it, *JARVIS* generates coarse-grained, monolithic queries that return broader result sets. The original query execution design decomposes these large SQL statements into modular sub-queries, which enables fine-grained and repeated querying that favors precision over recall. This behavior reinforces its role as a finer-grain filter.

When both modules are jointly enabled in *JARVIS*, their complementary mechanisms produce a distinct synergy. The query execution module enforces syntactic selectivity, while the processing module contributes semantic generalization through post-query refinement. Together, as Figure 16 shows, these mechanisms balance precision and recall, improving both selectivity and coverage, and ultimately leading to the best overall performance across all metrics.

Another interesting observation from Figure 16 regards the Thinking component. Even without this module (w/o Thinking), the system achieves precision comparable to the full *JARVIS* configuration. This indicates that the Thinking component primarily contributes to improvements in accuracy and recall by enforcing semantically grounded reasoning and enhancing the system’s ability to retrieve relevant but potentially under-specified data.

6.8 Processing Latency Breakdown

JARVIS comprises several software components that together form an end-to-end pipeline from user input to final response. While there is no strict latency requirement, the system must respond within a timeframe deemed “acceptable” for practical deployment as emphasized in **G4** of our design goals.

In Table 4, we present the latency incurred by each component of *JARVIS* for various queries involving different operations. The most notable observation is that the Expert-LLM accounts for the highest latency among all

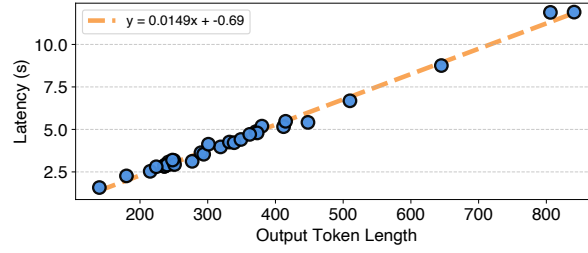


Fig. 17. Expert-LLM latency for varying token lengths, measured using RTX 4080 GPUs.

components, significantly more than the response-generating LLM. This is primarily because the Expert-LLM serves as the entry point to *JARVIS*, processing the full prompt and user input, which typically involves longer token sequences. In contrast, the response-generation LLM operates on more structured and concise inputs passed from other components within the Agent.

Queries that require some level of processing (Proc. 0 in the table) exhibit higher latency at both the Expert-LLM and the data processing stage compared to those that do not (Proc. X). While longer data processing time is expected, the increased latency at the Expert-LLM stems from the need to generate more complex internal execution instructions to support downstream Agent operations. A similar pattern is observed for queries with and without ambiguous terms (i.e., Ambig. 0 and Ambig. X), as input ambiguity typically necessitates more elaborate reasoning and planning by the Expert-LLM. Additionally, the processing latency is higher for Ambig. 0, which we attribute to our QA dataset containing more complex processing tasks for ambiguous queries.

We further analyze the Expert-LLM's latency with respect to its output token length in Figure 17. As the results show, there is a positive correlation between latency and token length, indicating that complex queries requiring longer and detailed execution instruction sets can lead to increased latency at the Expert-LLM.

While our LLMs offer a favorable balance between quality and latency suitable for our use case, we acknowledge the opportunity for future improvements. Exploring smaller, optimized model architectures (via quantization, distillation, or Mixture-of-Experts designs) may further reduce inference time without compromising performance. We leave this exploration to future work.

7 DISCUSSIONS

Based on our experience designing and evaluating *JARVIS*, we outline several discussion points that highlight limitations and open opportunities for future research.

Scope delineation from control and optimization. Recent work has demonstrated the potential of LLMs in HVAC control and optimization, including MPC- or reinforcement-learning-based formulations [1, 36, 48, 53, 64]. While these approaches aim to synthesize control actions under formal objectives and constraints, *JARVIS* is not designed as a control or decision-making system. Instead, we focus on enabling accurate data retrieval, semantic grounding of user-native queries, and faithful human interpretation of HVAC sensor data. We view *JARVIS* as complementary to such optimization pipelines, and exploring tighter integration between QA-driven interpretation and downstream control modules remains an important direction for future work.

Limitations of text-only interaction in spatial HVAC domains. Recent work has highlighted the potential of multimodal interfaces for HVAC analysis and control. For example, incorporating spatial or visual information such as floor plans or camera-based observations can provide richer situational awareness in spatially grounded tasks [9, 48]. While such approaches are important and can be effective, *JARVIS* adopts a text-based, conversational interface. Our design choice is motivated by the observation that many real-world HVAC interactions are expressed through ad-hoc, language-driven queries posed by human operators or occupants, and that existing

HVAC infrastructures primarily expose information through tabular sensor data rather than synchronized visual streams. As a result, the goal of *JARVIS* is not to reconstruct or reason over full spatial scenes, but to enable faithful retrieval, aggregation, and interpretation of sensor data that are already semantically meaningful to the system users. Nevertheless, we view multimodal and spatially grounded extensions as a natural next step, and exploring their integration with conversational QA remains an important direction for future work.

Failure cases and schema conformance. A recurring challenge we observed involves the Expert-LLM’s generation of malformed JSON outputs, despite targeted prompt tuning and fine-tuning efforts. These errors, such as missing or extraneous brackets, result in schema violations that cause execution failures. While relatively infrequent (2 out of 54 test cases in our evaluation), they highlight a broader issue when relying on free-form text generation for structured outputs. In this work, we address the problem using a regex-based post-processing patch; however, this approach remains brittle and reactive. An interesting direction for future work is to incorporate structured decoding mechanisms (e.g., constrained or programmatic decoding) to ensure that outputs conform to expected schemas by construction. This could improve system robustness and reduce downstream error handling complexity.

Human-in-the-loop error handling. When retrieval or generation failures occur in *JARVIS*, the system currently returns only the reason behind the error without offering actionable recovery guidance. While this is sufficient for diagnosing system-level issues, it places the burden of query reformulation entirely on the user. A promising direction for future work is to incorporate interactive recovery mechanisms that actively guide users toward successful queries. For example, the system could suggest semantically related questions that are known to be answerable, or prompt users to provide additional constraints when the original request is ambiguous or underspecified.

Expert-LLM as a semantic context manager. The Expert-LLM in *JARVIS* serves as a centralized module for acquiring and propagating domain-specific semantic context to downstream components. This centralized architecture offers broader applicability beyond HVAC, especially in modular frameworks that combine expert LLMs with other specialized components (including additional LLMs). By offloading semantic interpretation to a single module, the system reduces the need to re-engineer context representations across modules, lowering development overhead, improving scalability, and simplifying maintenance.

Trade-off between response richness and stability. As briefly discussed in Section 6.3, *JARVIS* shows a key limitation in response richness given that the response generation LLM is inclined to focus on the expectation template. While this bias reliably yields context-rich answers even from minimal semantic contexts, it can lessen the naturalness of the responses by curbing the model’s expressive freedom. In essence, this is a trade-off between producing flexible, conversational responses and delivering information-dense answers with lower risk of omission or hallucination [32], which is an intentional design choice we made to favor the latter. Balancing these competing objectives, perhaps by fine-tuning the response generator, is left for future work.

Practical HVAC workflow integration. Finally, while our current evaluation focuses on response accuracy and user-perceived quality, we acknowledge that demonstrating practical benefits within real-world building management workflows is essential for validating the utility of *JARVIS*. In future work, we plan to conduct a deployment-oriented user study involving facility managers and building operators to evaluate how *JARVIS* impacts operational metrics, such as the time to identify HVAC anomalies, frequency of manual query formulation, or perceived reduction in management effort. We believe such an evaluation is critical for understanding the broader utility of QA-driven HVAC interfaces in practice and will provide insight into how *JARVIS* can augment or streamline existing decision-making workflows. Despite so, we believe that establishing robust system performance is a necessary foundation for realizing downstream utility in deployment. Our findings thus

serve as a technical baseline for future work exploring longitudinal deployment and workflow integration in live building environments.

8 CONCLUSION

This work presented *JARVIS*, a LLM-based framework designed to enable truthful and context-aware question answering in HVAC sensing environments. By integrating structured sensor data with semantic reasoning through components such as metadata injection, intermediate thought generation, bottom-up planning and agent-side processing, *JARVIS* overcomes key limitations of traditional LLMs in dynamic, sensor-rich domains. Our evaluations, which included automated scoring via LLM-as-a-Judge, a real-world user study, and system performance analysis, demonstrates that *JARVIS* significantly outperforms baseline and ablation variants across most response quality metrics. Notably, the results show that each system component contributes meaningfully to both response quality and execution efficiency. Furthermore, our analysis reveals that *JARVIS* is capable of providing accurate and helpful responses even under underspecified or ambiguous conditions, showcasing its practical value in real-world deployments.

Acknowledgments

We thank the anonymous reviewers and associate editors for their efforts in providing constructive comments for the paper. This work was supported by the Korean Ministry of Science and ICT through the Institute for Information & Communication Technology Planning & Evaluation (IITP) via projects RS-2024-00434743 (YKCS Open RAN Global Collaboration Center) and RS-2024-00428780 (6G-Cloud R&E Open Hub Research Center). Correspondence should be made to both Pei Zhang and JeongGil Ko.

References

- [1] Ki Uhn Ahn, Deuk-Woo Kim, Hyun Mi Cho, and Chang-U Chae. 2023. Alternative approaches to hvac control of chat generative pre-trained transformer (chatgpt) for autonomous building system operations. *Buildings* 13, 11 (2023), 2680.
- [2] Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. 2024. Make your llm fully utilize the context. *Advances in Neural Information Processing Systems* 37 (2024), 62160–62188.
- [3] ASHRAE. 2024. Certified HVAC Designer (CHD). ASHRAE Official Website. Available from: <https://www.ashrae.org/professional-development/ashrae-certification/hvac-designer-certification>.
- [4] Anirban Basu, Yujia Wang, Yifan Li, et al. 2024. Seven Failure Points When Engineering a Retrieval Augmented Generation System. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 1–10.
- [5] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E Gonzalez, Carlos Guestrin, and Matei Zaharia. 2024. Text2sql is not enough: Unifying ai and databases with tag. *arXiv preprint arXiv:2408.14717* (2024).
- [6] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. 2025. Unleashing the potential of prompt engineering for large language models. *Patterns* 6, 6 (2025).
- [7] Yuxuan Cheng, Yuvraj Agarwal, and Rajesh Gupta. 2019. HVACMeter: Apportionment of HVAC Power to Thermal Zones and Air Handler Units. In *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '19)*. ACM, 61–70. doi:10.1145/3360322.3360840
- [8] Steffi Chern, Zhulin Hu, Yuqing Yang, Ethan Chern, Yuan Guo, Jiahe Jin, Binjie Wang, and Pengfei Liu. 2024. BeHonest: Benchmarking Honesty in Large Language Models. *arXiv preprint arXiv:2406.13261* (2024).
- [9] Eun Ji Choi, Bo Rang Park, Nam Hyeon Kim, and Jin Woo Moon. 2022. Effects of thermal comfort-driven control based on real-time clothing insulation estimated using an image-processing model. *Building and Environment* 223 (2022), 109438.
- [10] Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Tao He, Haotian Wang, Weihua Peng, Ming Liu, Bing Qin, and Ting Liu. 2024. Navigate through enigmatic labyrinth a survey of chain of thought reasoning: Advances, frontiers and future. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1173–1203.
- [11] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 150–158.
- [12] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD conference on knowledge*

- discovery and data mining*. 6491–6501.
- [13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).
 - [14] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
 - [15] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. 2024. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. *arXiv preprint arXiv:2410.12837* (2024).
 - [16] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
 - [17] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering* (2025).
 - [18] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *Iclr* 1, 2 (2022), 3.
 - [19] Gang Jiang, Zhihao Ma, Liang Zhang, and Jianli Chen. 2024. EPlus-LLM: A large language model-based computing platform for automated building energy modeling. *Applied Energy* 367 (2024), 124341.
 - [20] Gang Jiang, Zhihao Ma, Liang Zhang, and Jianli Chen. 2025. Prompt engineering to inform large language model in automated building energy modeling. *Energy* 316 (2025), 134548.
 - [21] Joonhee Lee, Hyunseung Shin, and Jeonggil Ko. 2026. IROS: A Dual-Process Architecture for Real-Time VLM-Based Indoor Navigation. *arXiv:2601.21506* [cs.RO] <https://arxiv.org/abs/2601.21506>
 - [22] Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15339–15353.
 - [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
 - [24] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, et al. 2025. From generation to judgment: Opportunities and challenges of llm-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2757–2791.
 - [25] Dan Li, Yuxun Zhou, Guoqiang Hu, and Costas J. Spanos. 2020. Handling Incomplete Sensor Measurements in Fault Detection and Diagnosis for Building HVAC Systems. *IEEE Transactions on Automation Science and Engineering* 17, 2 (2020), 833–846. doi:10.1109/TASE.2019.2948101
 - [26] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. *arXiv preprint arXiv:2412.05579* (2024).
 - [27] Junlong Li, Shichao Sun, Weizhe Yuan, Run-Ze Fan, Hai Zhao, and Pengfei Liu. 2023. Generative judge for evaluating alignment. *arXiv preprint arXiv:2310.05470* (2023).
 - [28] Siran Li, Linus Stenzel, Carsten Eickhoff, and Seyed Ali Bahrainian. 2025. Enhancing Retrieval-Augmented Generation: A Study of Best Practices. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*. 6705–6717.
 - [29] Wenwu Li, Xiangfeng Wang, Wenhao Li, and Bo Jin. 2025. A Survey of Automatic Prompt Engineering: An Optimization Perspective. *arXiv preprint arXiv:2502.11560* (2025).
 - [30] Yifan Li, Yichao Liu, Banghao Chen, and Shengxin Zhu. 2024. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *arXiv preprint arXiv:2402.07927* (2024).
 - [31] Zhiwei Li, Zhihao Wang, Banghao Chen, and Shengxin Zhu. 2023. On the Use of Virtual Knowledge Graphs to Improve Environmental Sensor Data Accessibility. *IEEE Transactions on Industrial Informatics* 20, 2 (2023), 1234–1245.
 - [32] Sheng-Chieh Lin, Luyu Gao, Barlas Oguz, Wenhan Xiong, Jimmy Lin, Wen-tau Yih, and Xilun Chen. 2024. Flame: Factuality-aware alignment for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 115588–115614.
 - [33] Kaiwei Liu, Bufang Yang, Lilin Xu, Yunqi Guo, Guoliang Xing, Xian Shuai, Xiaozhe Ren, Xin Jiang, and Zhenyu Yan. 2025. *TaskSense: A Translation-like Approach for Tasking Heterogeneous Sensor Systems with LLMs*. Association for Computing Machinery, New York, NY, USA, 213–225. <https://doi.org/10.1145/3715014.3722070>
 - [34] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chengguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 2511–2522.
 - [35] Jie Lu, Xiangning Tian, Chaobo Zhang, Yang Zhao, Jian Zhang, Wenkai Zhang, Chenxin Feng, Jianing He, Jiaxi Wang, and Fengtai He. 2024. Evaluation of large language models (LLMs) on the mastery of knowledge and skills in the heating, ventilation and air conditioning (HVAC) industry. *Energy and Built Environment* (2024).
 - [36] Gabriel Maher. 2025. LLMPC: Large Language Model Predictive Control. *arXiv preprint arXiv:2501.02486* (2025).
 - [37] Wes McKinney. 2010. Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*. 51–56.

- [38] NVIDIA. [n. d.]. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM> GitHub repository. Accessed: 2026-01-30.
- [39] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [40] JaeYeon Park, Kichang Lee, Sungmin Lee, Mi Zhang, and JeongGil Ko. 2023. Attfl: A personalized federated learning framework for time-series mobile and embedded sensor data processing. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 7, 3 (2023), 1–31.
- [41] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in neural information processing systems* 36 (2023), 36339–36348.
- [42] Muhammad Usman Rasheed et al. 2023. Leveraging SQL for Effective Data Acquisition in Wireless Sensor Networks: An Empirical Approach. *IEEE Sensors Journal* 23, 21 (2023), 23456–23467.
- [43] Jeff Reback, Wes McKinney, jbrockmendel, Joris Van den Bossche, Tom Augspurger, Simon Cloud, Kyle Hawkins, Gfyoung, Sinhrks, Michael Roeschke, Stephan Klein, Terji Petersen, Marc Tratner, Chang She, Mortada Mehryar Ayd, William Hoefler, Eyal Naveh, Samuel Garcia, Jeremy Schendel, Patrick Hayden, Brandon Saxton, and Joseph Gorelli. 2020. pandas-dev/pandas: Pandas. *Zenodo* (2020). doi:10.5281/zenodo.3509134
- [44] Attila Reiss and Didier Stricker. 2012. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th international symposium on wearable computers*. IEEE, 108–109.
- [45] LG AI Research, Soyoung An, Kyunghoon Bae, Eunbi Choi, Kibong Choi, Stanley Jungkyu Choi, Seokhee Hong, Junwon Hwang, Hyojin Jeon, Gerrard Jeongwon Jo, Hyunjik Jo, Jiyeon Jung, Yountae Jung, Hyosang Kim, Joonkee Kim, Seonghwan Kim, Soyeon Kim, Sunkyoung Kim, Yireun Kim, Yongil Kim, Youchul Kim, Edward Hwayoung Lee, Haeju Lee, Honglak Lee, Jinsik Lee, Kyungmin Lee, Woohyung Lim, Sangha Park, Sooyoun Park, Yongmin Park, Sihoon Yang, Heuiyeon Yeen, and Hyeongu Yun. 2024. EXAONE 3.5: Series of Large Language Models for Real-world Use Cases. arXiv:2412.04862 [cs.CL] <https://arxiv.org/abs/2412.04862>
- [46] Ellen Riloff and Michael Thelen. 2000. A rule-based question answering system for reading comprehension tests. In *ANLP-NAACL 2000 workshop: reading comprehension tests as evaluation for computer-based language understanding systems*.
- [47] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927* 1 (2024).
- [48] Tomoya Sawada, Masahiro Mizuno, Takaomi Hasegawa, Keiichi Yokoyama, and Mayuka Kono. 2025. Office-in-the-Loop: an investigation into Agentic AI for advanced building HVAC control systems. *Data-Centric Engineering* 6 (2025), e31.
- [49] Subhashini Venugopalan Shraman Pramanick, Rama Chellappa. 2024. SPIQA: A Dataset for Multimodal Question Answering on Scientific Papers. In *Neural Information Processing Systems Foundation, Inc. (NeurIPS)*.
- [50] Satwinder Singh and Himanshu Beniwal. 2022. A survey on near-human conversational agents. *Expert Systems with Applications* 202 (2022), 117116.
- [51] Kanghui Ning · Zijie Pan · Yu Liu · Yushan Jiang · James Zhang · Kashif Rasul · Anderson Schneider · Lintao Ma · Yuriy Nevmyvaka · Dongjin Song. 2024. TS-RAG: Retrieval-Augmented Generation based Time Series Foundation Models are Stronger Zero-Shot Forecaster. *Advances in Neural Information Processing Systems* (2024).
- [52] Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024. Dragin: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12991–13013.
- [53] S. Taheri, P. Hosseini, and A. Razban. 2022. Model predictive control of heating, ventilation, and air conditioning (HVAC) systems: A state-of-the-art review. *Journal of Building Engineering* 60 (2022), 105067. doi:10.1016/j.job.2022.105067
- [54] The Chill Brothers. 2025. *Smart Home and HVAC: How Energy Management Is Shaping Future Spending*. <https://thechillbrothers.com/smart-home-and-hvac-how-energy-management-is-shaping-future-spending/> By 2029, smart AC and heating systems are expected to generate \$8.31 billion..
- [55] Inc. Timescale. 2017. TimescaleDB: An Open-Source Time-Series SQL Database. <https://www.timescale.com/>.
- [56] Jiaan Wang, Yunlong Liang, Fandong Meng, Zengkui Sun, Haoxiang Shi, Zhixu Li, Jinan Xu, Jianfeng Qu, and Jie Zhou. 2023. Is chatgpt a good nlg evaluator? a preliminary study. In *Proceedings of the 4th New Frontiers in Summarization Workshop*. 1–11.
- [57] L. Wang, C. Ma, X. Feng, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18 (2024), 186345. doi:10.1007/s11704-024-40231-1
- [58] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2201.11903>
- [59] Arief Wicaksono, Farrel Izza Barunastra, Michael Chandra Jaya Giovani Suhari, and Fredy Jingga. 2024. Students level of trust in the use of AI chatbots in higher education: a quantitative analysis adopting technology acceptance model. In *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, Vol. 7. IEEE, 305–310.

- [60] Z. Xi, W. Chen, X. Guo, et al. 2025. The rise and potential of large language model based agents: a survey. *Science China Information Sciences* 68, 1 (2025), 121101. doi:10.1007/s11432-024-4222-0
- [61] Tong Xiao and Peng Xu. 2024. Exploring automated energy optimization with unstructured building data: A multi-agent based framework leveraging large language models. *Energy and Buildings* 322 (2024), 114691.
- [62] Shichao Xu, Yangyang Fu, Yixuan Wang, Zhuoran Yang, Chao Huang, Zheng O'Neill, Zhaoran Wang, and Qi Zhu. 2025. Efficient and assured reinforcement learning-based building HVAC control with heterogeneous expert-guided training. *Scientific Reports* 15, 1 (2025), 7677.
- [63] Bufang Yang, Siyang Jiang, Lilin Xu, Kaiwei Liu, Hai Li, Guoliang Xing, Hongkai Chen, Xiaofan Jiang, and Zhenyu Yan. 2024. DrHouse: An LLM-empowered Diagnostic Reasoning System through Harnessing Outcomes from Sensor Data and Expert Knowledge. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 8, 4, Article 153 (Nov. 2024), 29 pages. doi:10.1145/3699765
- [64] Donghao Yang, Aolang Wu, Tianyi Zhang, Li Zhang, Xiaoli Lian, Fang Liu, Yuming Ren, Jiaji Tian, and Xiaoyin Che. 2025. AutoPLC: Generating vendor-aware structured text for programmable logic controllers. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3593–3604.
- [65] Juan Yang, Jing Yu, and Shijing Wang. 2024. Heating ventilation air-conditioner system for multi-regional commercial buildings based on deep reinforcement learning. *Advanced Control for Applications: Engineering and Industrial Systems* 6, 2 (2024), e190.
- [66] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanell Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 3911–3921.
- [67] Xiaofan Yu, Lanxiang Hu, Benjamin Reichman, Dylan Chu, Rushil Chandrupatla, Xiyuan Zhang, Larry Heck, and Tajana S Rosing. 2025. Sensorchat: Answering qualitative and quantitative questions during long-term multimodal sensor interactions. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 9, 3 (2025), 1–35.
- [68] Beichen Zhang, Kun Zhou, Zheng Gong, Wayne Xin Zhao, and Ji-Rong Wen. 2021. ViCA: Combining visual, social, and task-oriented conversational AI in a healthcare setting. In *Proceedings of the 29th ACM International Conference on Multimedia*. 5174–5182. User study with 21 participants, evaluation of helpfulness and entertainment.
- [69] Jian Zhang, Chaobo Zhang, Jie Lu, and Yang Zhao. 2025. Domain-specific large language models for fault diagnosis of heating, ventilation, and air conditioning systems by labeled-data-supervised fine-tuning. *Applied Energy* 377 (2025), 124378.
- [70] Zhihan Zhang, Puvarin Thavikulwat, Alexander Le Metzger, Yuxuan Mei, Felix Hähnlein, Zachary Englhardt, Gregory D. Abowd, Shwetak Patel, Adriana Schulz, Tingyu Cheng, and Vikram Iyer. 2025. Living Sustainability: In-Context Interactive Environmental Impact Communication. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 9, 3, Article 153 (Sept. 2025), 42 pages. doi:10.1145/3749488
- [71] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2026. Retrieval-augmented generation for ai-generated content: A survey. *Data Science and Engineering* (2026), 1–29.
- [72] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [73] Xiaohu Zhu, Qian Li, Lizhen Cui, and Yongkang Liu. 2024. Large Language Model Enhanced Text-to-SQL Generation: A Survey. *arXiv preprint arXiv:2410.06011* (2024).